

```
cv=kfold,
scoring='accuracy')
grid.fit(X_train, High_train) grid.best_score_
```

Out[14]: 0.685

Hãy cùng xem xét cây đã được tia cành.

```
Trong [15]: ax = subplots(figsize=(12, 12))[1] best_ =
grid.best_estimator_ plot_tree(best_,

feature_names=feature_names, ax=ax);
```

Đây là một cái cây khá rậm rạp. Chúng ta có thể đếm lá, hoặc truy vấn `best_` thay thế.

```
Trong [16]: best_.tree_.n_leaves
```

Ra ngoài[16]: 30

Cây có 30 nút cuối cùng cho tỷ lệ lỗi kiểm định chéo thấp nhất, với độ chính xác là 68,5%. Cây được cắt tia này hoạt động tốt như thế nào trên tập dữ liệu thử nghiệm? Một lần nữa, chúng ta áp dụng hàm `predict()` .

```
Trong [17]: print(accuracy_score(High_test,
best_.predict(X_test))) confusion =
confusion_table(best_.predict(X_test), High_test)

lú lần
```

Out[17]: 0.72

	Sự thật	Không Có
Dự đoán		
KHÔNG	108	61
Đúng	10	21

Hiện tại, 72,0% số quan sát thử nghiệm được phân loại chính xác, con số này hơi tệ hơn so với lỗi của cây quyết định đầy đủ (với 35 lá). Vì vậy, phương pháp kiểm định chéo không giúp ích được nhiều ở đây; nó chỉ loại bỏ được 5 lá, với cái giá là lỗi tệ hơn một chút. Kết quả này sẽ thay đổi nếu chúng ta thay đổi các hạt giống số ngẫu nhiên ở trên; mặc dù kiểm định chéo cung cấp một cách tiếp cận không thiên vị đối với việc lựa chọn mô hình, nhưng nó vẫn có sự biến động.

8.3.2 Xây dựng cây hồi quy

Ở đây, chúng ta sẽ xây dựng một cây hồi quy cho tập dữ liệu `Boston` . Các bước thực hiện tương tự như đối với cây phân loại.

```
Trong [18]: Boston = load_data("Boston") model =
MS(Boston.columns.drop('medv'), intercept=False)
D = model.fit_transform(Boston) feature_names =
list(D.columns)
X = np.asarray(D)
```


8.3.3 Phương pháp đóng gói và rừng ngẫu nhiên

Ở đây, chúng ta áp dụng phương pháp bagging và rừng ngẫu nhiên cho tập dữ liệu `Boston`, sử dụng hàm `RandomForestRegressor()` từ gói `sklearn.ensemble`. Nhớ lại rằng bagging chỉ đơn giản là một trường hợp đặc biệt của rừng ngẫu nhiên với $m = p$. Do đó, hàm `RandomForestRegressor()` có thể được sử dụng để thực hiện cả bagging và rừng ngẫu nhiên. Chúng ta bắt đầu với bagging.

Rừng ngẫu nhiên
Bộ hồi quy()
sklearn.
dàn nhạc

```
Trong [24]: bag_boston = RF(max_features=X_train.shape[1], random_state=0)
           bag_boston.fit(X_train, y_train)
```

```
Out[24]: RandomForestRegressor(max_features=12, random_state=0)
```

Tham số `max_features` cho biết tất cả 12 biến dự đoán nên được xem xét cho mỗi lần phân nhánh của cây quyết định – nói cách khác, nên thực hiện kỹ thuật bagging. Mô hình bagging này hoạt động tốt như thế nào trên tập dữ liệu kiểm thử?

```
Trong [25]: ax = subplots(figsize=(8,8))[1]
           y_hat_bag = bag_boston.predict(X_test)
           ax.scatter(y_hat_bag, y_test)
           np.mean((y_test - y_hat_bag)**2)
```

```
Ra[25]: 14.63
```

Sai số bình phương trung bình (MSE) trên tập dữ liệu kiểm thử liên quan đến cây hồi quy kết hợp (bagged regression tree) là 14,63, bằng khoảng một nửa so với sai số thu được khi sử dụng cây đơn được cắt tỉa tối ưu. Chúng ta có thể thay đổi số lượng cây được tạo ra từ giá trị mặc định là 100 bằng cách sử dụng đối số `n_estimators`:

```
Trong [26]: bag_boston = RF(max_features=X_train.shape[1],
                           n_estimators=500,
                           random_state=0).fit(X_train, y_train)
           y_hat_bag = bag_boston.predict(X_test)
           np.mean((y_test - y_hat_bag)**2)
```

```
Ra[26]: 14.61
```

Không có nhiều thay đổi. Thuật toán Bagging và Random Forests không thể bị quá khớp (overfitting) khi tăng số lượng cây, nhưng có thể bị thiếu khớp (underfitting) nếu số lượng cây quá ít.

Việc xây dựng một rừng ngẫu nhiên diễn ra hoàn toàn giống nhau, ngoại trừ việc chúng ta sử dụng giá trị nhỏ hơn cho đối số `max_features`. Theo mặc định, `RandomForestRegressor()` sử dụng p biến khi xây dựng một rừng ngẫu nhiên gồm các cây hồi quy (tức là mặc định sử dụng phương pháp bagging), và `RandomForestClassifier()` sử dụng $\sqrt{p}$ biến khi xây dựng một rừng ngẫu nhiên gồm các cây phân loại. Ở đây chúng tôi sử dụng `max_features=6`.

```
Trong [27]: RF_boston = RF(max_features=6,
                          random_state=0).fit(X_train, y_train)
           y_hat_RF = RF_boston.predict(X_test)
           np.mean((y_test - y_hat_RF)**2)
```

```
Ra ngoài[27]: 20.04
```

MSE trên tập dữ liệu kiểm thử là 20,04; điều này cho thấy rừng ngẫu nhiên hoạt động kém hơn một chút so với bagging trong trường hợp này. Bằng cách trích xuất các giá trị `feature_importances_` từ mô hình đã được huấn luyện, chúng ta có thể xem tầm quan trọng của từng biến.

```
Trong [28]: feature_imp = pd.DataFrame(  
            {'tầm quan trọng': RF_boston.feature_importances_,  
             chỉ mục=tên_tính_năng)  
            feature_imp.sort_values(by='importance', ascending=False)
```

```
Ra ngoài[28]:          tầm quan trọng  
lstat          0.368683  
rm             0.333842  
ptratio        0.057306  
indus          0.053303  
tối phạm      0.052426  
sự            0.042493  
nox           0.034410  
thuế          0.024327  
tuổi tác      0.022368  
rad           0.005048  
kẽm           0.003238  
chas          0.002557
```

Đây là thước đo tương đối về tổng mức giảm tạp chất trong nút mạng.
từ các lần phân tách dựa trên biến đó, được tính trung bình trên tất cả các cây (điều này đã được vẽ biểu đồ trong Hình 8.9 minh họa mô hình phù hợp với dữ liệu về tìm .
Kết quả cho thấy rằng trên tất cả các cây được xem xét trong quá trình ngẫu nhiên rừng, mức độ giàu có của cộng đồng (lstat) và diện tích nhà ở (rm)
Đây là hai biến số quan trọng nhất.

8.3.4 Tăng cường

Ở đây, chúng ta sử dụng GradientBoostingRegressor() từ sklearn.ensemble để huấn luyện cây hồi quy tăng cường cho tập dữ liệu Boston . Đối với phân loại, chúng ta sẽ...
Sử dụng GradientBoostingClassifier(). Tham số n_estimators=5000 cho biết chúng ta muốn 5000 cây, và tùy chọn max_depth=3 giới hạn độ sâu của mỗi cây. Tham số learning_rate là λ đã được đề cập trước đó trong phần Mô tả về việc tăng cường hiệu suất.

Độ dốc
Tăng cường
Bộ hồi quy()
Độ dốc
Tăng cường
Bộ phân loại()

```
Trong [29]: boost_boston = GBR(n_estimators=5000,  
                               tốc độ học tập = 0.001,  
                               max_depth=3,  
                               random_state=0)  
boost_boston.fit(X_train, y_train)
```

Chúng ta có thể thấy lỗi huấn luyện giảm dần như thế nào khi sử dụng thuộc tính `train\_score\_` . Để hiểu rõ hơn về sự giảm của lỗi kiểm thử, chúng ta có thể sử dụng...
Phương thức staged_predict() được sử dụng để lấy các giá trị dự đoán dọc theo đường dẫn.

```
Trong [30]: test_error = np.zeros_like(boost_boston.train_score_)  
for idx, y_ in enumerate(boost_boston.staged_predict(X_test)):  
    test_error[idx] = np.mean((y_test - y_)**2)  
  
plot_idx = np.arange(boost_boston.train_score_.shape[0])  
ax = subplots(figsize=(8,8))[1]  
ax.plot(plot_idx,  
        boost_boston.train_score_,  
        'b',  
        nhãn='Đào tạo')
```

```
ax.plot(plot_idx,
        test_error,
        'r',
        label='Test')
ax.legend();
```

Giờ đây, chúng ta sử dụng mô hình được cải tiến để dự đoán `medv` trên tập dữ liệu thử nghiệm:

```
Trong [31]: y_hat_boost = boost_boston.predict(X_test); np.mean((y_test
- y_hat_boost)**2)
```

Ra ngoài[31]: 14.48

MSE kiểm tra thu được là 14,48, tương tự như MSE kiểm tra cho bagging. Nếu muốn, chúng ta có thể thực hiện boosting với giá trị khác của tham số co rút λ trong (8.10). Giá trị mặc định là 0,001, nhưng giá trị này có thể dễ dàng thay đổi. Ở đây ta lấy $\lambda = 0,2$.

```
Trong [32]: boost_boston = GBR(n_estimators=5000,
                             learning_rate=0.2,
                             max_depth=3,
                             random_state=0)
boost_boston.fit(X_train, y_train)
y_hat_boost
= boost_boston.predict(X_test); np.mean((y_test -
y_hat_boost)**2)
```

Ra ngoài[32]: 14.50

Trong trường hợp này, việc sử dụng $\lambda = 0,2$ dẫn đến giá trị MSE kiểm định gần như tương tự như khi sử dụng $\lambda = 0,001$.

8.3.5 Cây hồi quy cộng tính Bayes

Trong phần này, chúng tôi trình bày cách triển khai hàm BART bằng Python, được tìm thấy trong gói ISLP.bart. Chúng tôi sẽ áp dụng mô hình này cho tập dữ liệu nhà ở Boston. Trình ước lượng BART() này được thiết kế cho các biến kết quả định lượng, mặc dù cũng có các cách triển khai khác để áp dụng mô hình logistic và probit cho các biến kết quả phân loại.

```
Trong [33]: bart_boston = BART(random_state=0, burnin=5, ndraw=15) bart_boston.fit(X_train,
y_train)
```

Out[33]: BART(burnin=5, ndraw=15, random_state=0)

Trên tập dữ liệu này, với việc chia thành tập dữ liệu kiểm thử và huấn luyện, chúng ta thấy rằng lỗi kiểm thử của BART tương tự như lỗi của thuật toán rừng ngẫu nhiên.

```
Trong [34]: yhat_test = bart_boston.predict(X_test.astype(np.float32)) np.mean((y_test -
yhat_test)**2)
```

Ra[34]: 20,92

Chúng ta có thể kiểm tra số lần mỗi biến xuất hiện trong tập hợp các cây quyết định. Điều này cung cấp một bản tóm tắt tương tự như biểu đồ tầm quan trọng của biến đối với thuật toán boosting và random forests.

```
Trong [35]: var_inclusion = pd.Series(bart_boston.variable_inclusion_.mean(0),
                                   chỉ mục=D.cột)
var_inclusion
```

```
Ra ngoài[35]:   tội phạm    25.333333
               kềm      27.000.000
indus         21.266667
chas          20.466667
nox           25.400000
rm            32.400000
tuổi          26.133333
tác           25.666667
rad           24.666667
thuế          23.933333
ptratio       25.000.000
lstat         31.866667
kiểu dữ liệu: float64
```

8.4 Bài tập

Khái niệm

1. Hãy vẽ một ví dụ (do bạn tự nghĩ ra) về cách phân vùng không gian đặc trưng hai chiều có thể thu được từ phép toán nhị phân đệ quy. chia tách. Ví dụ của bạn nên chứa ít nhất sáu vùng. Hãy vẽ một cây quyết định tương ứng với phân vùng này. Hãy chắc chắn ghi nhận tất cả các khía cạnh của hình vẽ, bao gồm các vùng $R_1, R_2, \dots$, và các điểm cắt. $t_1, t_2, \dots$, và cứ thế tiếp tục.

Gợi ý: Kết quả của bạn sẽ trông giống như Hình 8.1 và 8.2.

2. Mục 8.2.3 có đề cập đến việc tăng cường sử dụng cây có độ sâu một. (hoặc gốc cây) dẫn đến một mô hình cộng tính : tức là, một mô hình có dạng

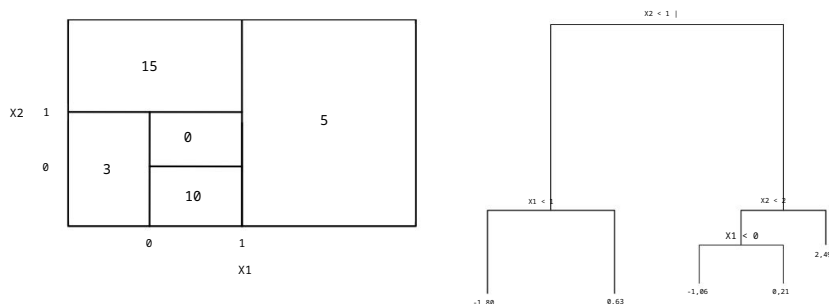
$$f(X) = \sum_{j=1}^P f_j(X_j).$$

Hãy giải thích tại sao lại như vậy. Bạn có thể bắt đầu với (8.12) trong Thuật toán 8.2.

3. Hãy xem xét chỉ số Gini, lỗi phân loại và entropy trong một ví dụ đơn giản. Thiết lập phân loại với hai lớp. Tạo một đồ thị duy nhất hiển thị mỗi đại lượng này như một hàm của p^m_1 . Trục x nên hiển thị p^m_1 , có giá trị từ 0 đến 1, và trục y nên hiển thị Giá trị của chỉ số Gini, lỗi phân loại và entropy.

Gợi ý: Trong trường hợp có hai lớp, $p^m_1 = 1 - p^m_2$. Bạn có thể tạo ra Tôi vẽ biểu đồ này bằng tay, nhưng sẽ dễ dàng hơn nhiều nếu dùng R.

4. Câu hỏi này liên quan đến các đồ thị trong Hình 8.14.



HÌNH 8.14. Bên trái: Phân vùng không gian dự đoán tương ứng với Bài tập 4a. Bên phải: Cây tương ứng với Bài tập 4b.

(a) Vẽ sơ đồ cây tương ứng với sự phân chia không gian biến dự đoán được minh họa trong hình bên trái của Hình 8.14. Các số bên trong ô biểu thị giá trị trung bình của Y trong mỗi vùng. (b) Tạo một sơ đồ tương tự như hình bên trái của Hình 8.14, sử dụng sơ đồ cây được minh họa trong hình bên phải của cùng hình đó. Bạn nên chia không gian biến dự đoán thành các vùng chính xác và chỉ ra giá trị trung bình cho mỗi vùng.

5. Giả sử chúng ta tạo ra mười mẫu bootstrap từ một tập dữ liệu chứa các lớp màu đỏ và xanh lá cây. Sau đó, chúng ta áp dụng cây phân loại cho mỗi mẫu bootstrap và, với một giá trị X cụ thể, tạo ra 10 ước tính của $P(\text{Lớp là Đỏ}|X)$:

0,1, 0,15, 0,2, 0,2, 0,55, 0,6, 0,6, 0,65, 0,7 và 0,75.

Có hai cách phổ biến để kết hợp các kết quả này lại với nhau thành một dự đoán lớp duy nhất. Một là phương pháp bỏ phiếu đa số được thảo luận trong chương này. Cách thứ hai là phân loại dựa trên xác suất trung bình. Trong ví dụ này, kết quả phân loại cuối cùng theo mỗi cách tiếp cận là gì?

6. Hãy trình bày chi tiết thuật toán được sử dụng để xây dựng cây hồi quy.

Đã áp dụng

7. Trong Mục 8.3.3, chúng ta đã áp dụng thuật toán rừng ngẫu nhiên cho tập dữ liệu Boston với `max_features = 6` và `n_estimators = 100` và `n_estimators = 500`. Hãy tạo một biểu đồ hiển thị lỗi kiểm thử thu được từ thuật toán rừng ngẫu nhiên trên tập dữ liệu này với phạm vi giá trị rộng hơn cho `max_features` và `n_estimators`. Bạn có thể mô phỏng biểu đồ của mình theo Hình 8.10. Mô tả các kết quả thu được.
8. Trong phòng thí nghiệm, cây phân loại đã được áp dụng cho tập dữ liệu Ghé ô tô sau khi chuyển đổi Doanh số thành biến phản hồi định tính. Bây giờ chúng ta sẽ tìm cách dự đoán Doanh số bằng cách sử dụng cây hồi quy và các phương pháp liên quan, coi phản hồi là một biến định lượng.

- (a) Chia tập dữ liệu thành tập huấn luyện và tập kiểm tra. (b) Xây dựng

cây hồi quy trên tập huấn luyện. Vẽ đồ thị cây và giải thích kết quả. Bạn thu được giá trị MSE kiểm tra là bao nhiêu?

- (c) Sử dụng phương pháp kiểm định chéo để xác định mức độ phức tạp tối ưu của cây. Việc cắt tỉa cây có cải thiện MSE kiểm thử không?

- (d) Sử dụng phương pháp bagging để phân tích dữ liệu này. Bạn thu được MSE kiểm thử là bao nhiêu? Sử dụng các giá trị `feature_importance_` để xác định biến nào quan trọng nhất. (e) Sử dụng rừng ngẫu nhiên để phân tích dữ liệu này.

Bạn thu được MSE kiểm thử là bao nhiêu? Sử dụng các giá trị `feature_importance_` để xác định biến nào quan trọng nhất. Mô tả ảnh hưởng của m , số lượng biến được xem xét ở mỗi lần chia, đến tỷ lệ lỗi thu được.

- (f) Bây giờ hãy phân tích dữ liệu bằng phương pháp BART và báo cáo kết quả của bạn.

9. Bài toán này liên quan đến tập dữ liệu `OJ`, một phần của `ISLP`.

bầu kiện.

- (a) Tạo một tập huấn luyện chứa một mẫu ngẫu nhiên gồm 800 quan sát và một tập kiểm tra chứa các quan sát còn lại. (b) Xây dựng một cây quyết định trên dữ liệu huấn

luyện, với biến `Purchase` là biến phản hồi và các biến khác là biến dự đoán. Tỷ lệ lỗi huấn luyện là bao nhiêu?

- (c) Vẽ sơ đồ cây và giải thích kết quả. Có bao nhiêu Cây này có những nút cuối cùng nào?

- (d) Sử dụng hàm `export_tree()` để tạo bản tóm tắt dạng văn bản của cây đã được huấn luyện. Chọn một trong các nút cuối cùng và giải thích thông tin được hiển thị.

- (e) Dự đoán phản hồi trên dữ liệu thử nghiệm và tạo ma trận nhầm lẫn so sánh nhãn thử nghiệm với nhãn thử nghiệm được dự đoán. Tỷ lệ lỗi kiểm thử là bao nhiêu?

- (f) Sử dụng phương pháp kiểm định chéo trên tập huấn luyện để xác định Kích thước cây tối ưu.

- (g) Tạo biểu đồ với kích thước cây trên trục x và kiểm định chéo. Tỷ lệ lỗi phân loại trên trục y .

- (h) Kích thước cây nào tương ứng với phân loại được kiểm định chéo thấp nhất? Tỷ lệ lỗi phân loại?

- (i) Tạo một cây được cắt tỉa tương ứng với kích thước cây tối ưu thu được bằng phương pháp kiểm định chéo. Nếu kiểm định chéo không dẫn đến việc lựa chọn một cây được cắt tỉa, thì hãy tạo một cây được cắt tỉa với năm nút cuối.

- (j) So sánh tỷ lệ lỗi huấn luyện giữa cây đã tỉa và cây chưa tỉa. Cây nào có tỷ lệ lỗi cao hơn?

- (k) So sánh tỷ lệ lỗi kiểm thử giữa cây đã tỉa cành và cây chưa tỉa cành. Cây nào có tỷ lệ lỗi cao hơn?

10. Giờ đây, chúng tôi sử dụng thuật toán boosting để dự đoán mức lương trong tập dữ liệu *Hitters*.

- (a) Loại bỏ các quan sát mà thông tin về lương không được biết, sau đó biến đổi logarit các mức lương.
- (b) Tạo một tập huấn luyện gồm 200 quan sát đầu tiên và một tập kiểm tra gồm các quan sát còn lại.
- (c) Thực hiện thuật toán boosting trên tập huấn luyện với 1.000 cây quyết định cho một loạt các giá trị của tham số co rút λ . Tạo một đồ thị với các giá trị co rút khác nhau trên trục x và MSE tương ứng của tập huấn luyện trên trục y.

- (d) Vẽ đồ thị với các giá trị co rút khác nhau trên trục x và MSE của tập dữ liệu kiểm tra tương ứng trên trục y. (e) So sánh MSE kiểm tra

của phương pháp boosting với MSE kiểm tra thu được từ việc áp dụng hai phương pháp hồi quy đã trình bày trong Chương 3 và 6. (f) Biến nào có vẻ là biến dự báo quan trọng nhất trong

Mô hình được nâng cấp?

- (g) Bây giờ hãy áp dụng bagging cho tập huấn luyện. MSE của tập kiểm tra với phương pháp này là bao nhiêu?

11. Câu hỏi này sử dụng tập dữ liệu *Caravan*.

- (a) Tạo một tập huấn luyện gồm 1.000 quan sát đầu tiên và một tập kiểm tra gồm các quan sát còn lại.
- (b) Hãy huấn luyện một mô hình boosting trên tập dữ liệu huấn luyện với biến *Purchase* là biến phản hồi và các biến khác là biến dự đoán. Sử dụng 1.000 cây quyết định và giá trị co rút là 0,01. Biến dự đoán nào có vẻ quan trọng nhất?

- (c) Sử dụng mô hình tăng cường để dự đoán phản hồi trên dữ liệu thử nghiệm. Dự đoán một người sẽ mua hàng nếu xác suất mua hàng ước tính lớn hơn 20%. Lập ma trận nhầm lẫn. Tỷ lệ người được dự đoán sẽ mua hàng thực sự mua hàng là bao nhiêu? So sánh kết quả này với kết quả thu được khi áp dụng thuật toán KNN hoặc hồi quy logistic cho tập dữ liệu này như thế nào?

12. Áp dụng boosting, bagging, random forests và BART cho một tập dữ liệu bạn chọn. Hãy chắc chắn rằng bạn đã huấn luyện các mô hình trên tập dữ liệu huấn luyện và đánh giá hiệu suất của chúng trên tập dữ liệu kiểm tra. Độ chính xác của kết quả so với các phương pháp đơn giản như hồi quy tuyến tính hoặc hồi quy logistic như thế nào? Phương pháp nào cho hiệu suất tốt nhất?

9

Máy hỗ trợ vectơ



Trong chương này, chúng ta sẽ thảo luận về máy vectơ hỗ trợ (SVM), một phương pháp phân loại được phát triển trong cộng đồng khoa học máy tính vào những năm 1990 và đã trở nên phổ biến kể từ đó. SVM đã được chứng minh là hoạt động tốt trong nhiều môi trường khác nhau và thường được coi là một trong những bộ phân loại "sẵn sàng sử dụng" tốt nhất.

Máy vectơ hỗ trợ (SVM) là sự tổng quát hóa của một bộ phân loại đơn giản và trực quan gọi là bộ phân loại biên độ tối đa, mà chúng ta sẽ giới thiệu trong Phần 9.1. Mặc dù thanh lịch và đơn giản, chúng ta sẽ thấy rằng bộ phân loại này không thể áp dụng cho hầu hết các tập dữ liệu, vì nó yêu cầu các lớp phải được phân tách bởi một ranh giới tuyến tính. Trong Phần 9.2, chúng ta giới thiệu bộ phân loại vectơ hỗ trợ, một phần mở rộng của bộ phân loại biên độ tối đa có thể được áp dụng trong phạm vi trường hợp rộng hơn. Phần 9.3 giới thiệu máy vectơ hỗ trợ, là một phần mở rộng hơn nữa của bộ phân loại vectơ hỗ trợ để phù hợp với các ranh giới lớp phi tuyến tính. Máy vectơ hỗ trợ được thiết kế cho thiết lập phân loại nhị phân trong đó có hai lớp; trong Phần 9.4, chúng ta thảo luận về các phần mở rộng của máy vectơ hỗ trợ cho trường hợp có nhiều hơn hai lớp. Trong Phần 9.5, chúng ta thảo luận về mối liên hệ chặt chẽ giữa máy vectơ hỗ trợ và các phương pháp thống kê khác như hồi quy logistic.

Người ta thường hay gọi chung bộ phân loại biên độ tối đa, bộ phân loại vectơ hỗ trợ và máy vectơ hỗ trợ là "máy vectơ hỗ trợ". Để tránh nhầm lẫn, chúng ta sẽ phân biệt rõ ràng giữa ba khái niệm này trong chương này.

9.1 Bộ phân loại biên độ tối đa

Trong phần này, chúng ta sẽ định nghĩa siêu mặt phẳng và giới thiệu khái niệm siêu mặt phẳng phân tách tối ưu.

9.1.1 Siêu mặt phẳng là gì?

Trong không gian p chiều, siêu mặt phẳng là một không gian con phẳng affine có chiều siêu mặt phẳng là $p - 1$.¹ Ví dụ, trong không gian hai chiều, siêu mặt phẳng là một không gian con phẳng một chiều—nói cách khác, là một đường thẳng. Trong không gian ba chiều, siêu mặt phẳng là một không gian con phẳng hai chiều—tức là một mặt phẳng. Trong không gian $p > 3$ chiều, việc hình dung một siêu mặt phẳng có thể khó khăn, nhưng khái niệm về không gian con phẳng ($p - 1$) chiều vẫn áp dụng được.

Định nghĩa toán học của siêu mặt phẳng khá đơn giản. Trong không gian hai chiều, siêu mặt phẳng được định nghĩa bởi phương trình sau:

$$\beta_0 + \beta_1 x_1 + \beta_2 x_2 = 0 \quad (9.1)$$

với các tham số β_0 , β_1 và β_2 . Khi chúng ta nói rằng (9.1) "xác định" siêu mặt phẳng, chúng ta có nghĩa là bất kỳ $X = (x_1, x_2)^T$ nào mà (9.1) đúng đều là một điểm trên siêu mặt phẳng. Lưu ý rằng (9.1) chỉ đơn giản là phương trình của một đường thẳng, vì thực sự trong không gian hai chiều, một siêu mặt phẳng là một đường thẳng.

Phương trình 9.1 có thể dễ dàng mở rộng sang thiết lập p chiều:

$$\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p = 0 \quad (9.2)$$

xác định một siêu mặt phẳng p chiều, một lần nữa theo nghĩa là nếu một điểm $X = (x_1, x_2, \dots, x_p)^T$ trong không gian p chiều (tức là một vectơ có độ dài p) thỏa mãn (9.2), thì X nằm trên siêu mặt phẳng.

Bây giờ, giả sử rằng X không thỏa mãn (9.2); thay vào đó,

$$\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p > 0. \quad (9.3)$$

Điều này cho chúng ta biết rằng X nằm ở một phía của siêu mặt phẳng. Mặt khác, nếu

$$\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p < 0, \quad (9.4)$$

Khi đó X nằm ở phía bên kia của siêu mặt phẳng. Vì vậy, ta có thể coi siêu mặt phẳng như chia không gian p chiều thành hai nửa. Người ta có thể dễ dàng xác định một điểm nằm ở phía nào của siêu mặt phẳng bằng cách đơn giản tính dấu của vế trái của (9.2). Một siêu mặt phẳng trong không gian hai chiều được thể hiện trong Hình 9.1.

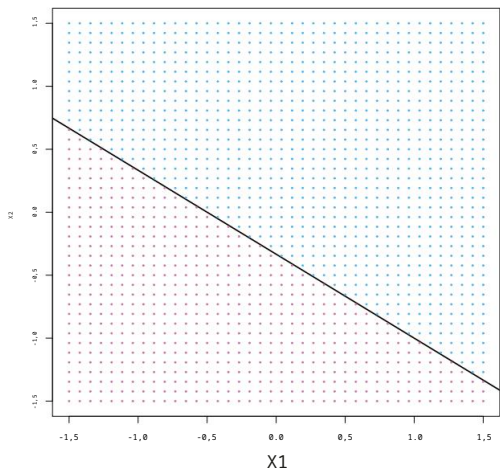
9.1.2 Phân loại bằng cách sử dụng siêu mặt phẳng phân tách

Bây giờ giả sử chúng ta có một ma trận dữ liệu X kích thước $n \times p$ bao gồm n quan sát huấn luyện trong không gian p chiều,

$$X = \begin{bmatrix} x_{11} & \dots & x_{n1} \\ \vdots & & \vdots \\ x_{1p} & \dots & x_{np} \end{bmatrix}, \quad (9.5)$$

và những quan sát này được chia thành hai lớp—tức là, $y_1, \dots, y_n \in \{ -1, 1 \}$ trong đó -1 đại diện cho một lớp và 1 đại diện cho lớp còn lại. Chúng ta cũng có một

1. Từ " affine " chỉ ra rằng không gian con không nhất thiết phải đi qua gốc tọa độ.



HÌNH 9.1. Mặt phẳng siêu phẳng $1+2X_1+3X_2=0$ được hiển thị. Vùng màu xanh lam là tập hợp các điểm mà $1+2X_1+3X_2>0$, và vùng màu tím là tập hợp các điểm đó. các điểm mà $1+2X_1+3X_2<0$.

quan sát thử nghiệm, một vectơ p của các đặc điểm quan sát được $x = x_1 \dots x_p$ của chúng tôi. Mục tiêu là phát triển một bộ phân loại dựa trên dữ liệu huấn luyện để có thể phân loại chính xác. phân loại quan sát thử nghiệm bằng cách sử dụng các phép đo đặc trưng của nó. Chúng ta đã thấy Có một số phương pháp tiếp cận cho nhiệm vụ này, chẳng hạn như phân tích phân biệt tuyến tính. và hồi quy logistic trong Chương 4, và cây phân loại, bagging, và sự thúc đẩy trong Chương 8. Bây giờ chúng ta sẽ xem xét một cách tiếp cận mới dựa trên Khái niệm về siêu mặt phẳng phân tách.

Giả sử có thể xây dựng một siêu mặt phẳng phân tách... tách biệt siêu mặt phẳng Các quan sát huấn luyện hoàn toàn phù hợp với nhãn lớp của chúng. Ví dụ Hình ảnh của ba siêu mặt phẳng phân tách như vậy được hiển thị ở bảng bên trái. Hình 9.2. Chúng ta có thể dán nhãn các quan sát từ lớp màu xanh lam là $y_i = 1$ và những người thuộc lớp màu tím có $y_i = -1$. Khi đó, một siêu mặt phẳng phân tách có tài sản đó

$$\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip} > 0 \text{ nếu } y_i = 1, \tag{9.6}$$

Và

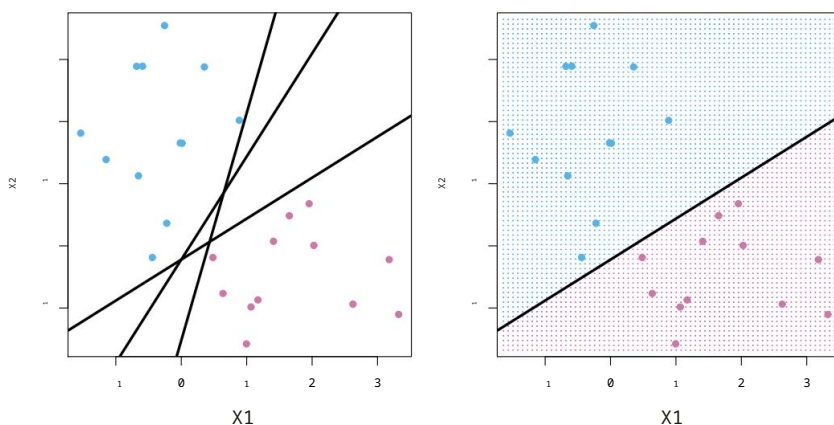
$$\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip} < 0 \text{ nếu } y_i = -1. \tag{9.7}$$

Tương đương, một siêu mặt phẳng phân tách có đặc tính là

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip}) > 0 \tag{9.8}$$

với mọi $i = 1, \dots, n$.

Nếu tồn tại một siêu mặt phẳng phân tách, chúng ta có thể sử dụng nó để xây dựng một cấu trúc rất tự nhiên. bộ phân loại: một quan sát thử nghiệm được gán vào một lớp tùy thuộc vào phía nào của siêu mặt phẳng mà nó nằm. Hình bên phải của Hình 9.2 cho thấy một ví dụ về bộ phân loại như vậy. Nghĩa là, chúng ta phân loại quan sát thử nghiệm x^* dựa trên dấu của $f(x) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p$. Nếu $f(x)$ dương, sau đó chúng ta gán quan sát thử nghiệm vào lớp 1, và nếu $f(x)$ là âm thì chúng ta gán nó vào lớp -1. Chúng ta cũng có thể sử dụng độ lớn của $f(x)$. Nếu



HÌNH 9.2. Bên trái: Có hai loại quan sát, được hiển thị bằng màu xanh lam và màu tím, mỗi cái có số đo trên hai biến. Ba cái tách biệt

Các siêu mặt phẳng, trong số nhiều siêu mặt phẳng khả thi, được hiển thị bằng màu đen. Bên phải: Một siêu mặt phẳng phân tách được hiển thị bằng màu đen. Lưới màu xanh lam và tím biểu thị quy tắc quyết định. Được tạo ra bởi một bộ phân loại dựa trên siêu mặt phẳng phân tách này: một quan sát thử nghiệm mà Những mục nằm trong phần màu xanh lam của lưới sẽ được xếp vào lớp màu xanh lam, và một bài kiểm tra sẽ được thực hiện. Quan sát nào rơi vào phần màu tím của lưới sẽ được gán cho... Lớp màu tím.

Nếu $f(x)$ khác không, điều đó có nghĩa là x nằm cách xa siêu mặt phẳng.

và do đó chúng ta có thể tự tin về bài tập trên lớp cho môn x^* . Mặt khác

Mặt khác, nếu $f(x)$ gần bằng 0, thì x nằm gần siêu mặt phẳng, và do đó

Chúng tôi không chắc chắn lắm về việc phân loại cho x^* . Điều này không có gì đáng ngạc nhiên, và Như chúng ta thấy trong Hình 9.2, một bộ phân loại dựa trên siêu mặt phẳng phân tách.

dẫn đến một ranh giới quyết định tuyến tính.

9.1.3 Bộ phân loại biên độ tối đa

Nói chung, nếu dữ liệu của chúng ta có thể được phân tách hoàn hảo bằng một siêu mặt phẳng, thì Trên thực tế sẽ tồn tại vô số siêu mặt phẳng như vậy. Điều này là

bởi vì một siêu mặt phẳng phân tách nhất định thường có thể được dịch chuyển lên một chút hoặc hạ xuống hoặc xoay tròn mà không tiếp xúc với bất kỳ đối tượng quan sát nào.

Hình bên trái hiển thị ba siêu mặt phẳng phân tách khả thi.

của Hình 9.2. Để xây dựng một bộ phân loại dựa trên sự phân tách

siêu phẳng, chúng ta phải có một cách hợp lý để quyết định xem trong vô số cái nào các siêu mặt phẳng phân tách khả thi có thể sử dụng.

Một lựa chọn tự nhiên là siêu mặt phẳng có biên độ tối đa (còn được gọi là siêu mặt phẳng phân tách tối ưu), đó là siêu mặt phẳng phân tách mà

là điểm xa nhất so với các quan sát huấn luyện. Tức là, chúng ta có thể tính toán

Khoảng cách (vuông góc) từ mỗi điểm quan sát huấn luyện đến một siêu mặt phẳng phân tách nhất định; khoảng cách nhỏ nhất như vậy là khoảng cách tối thiểu từ

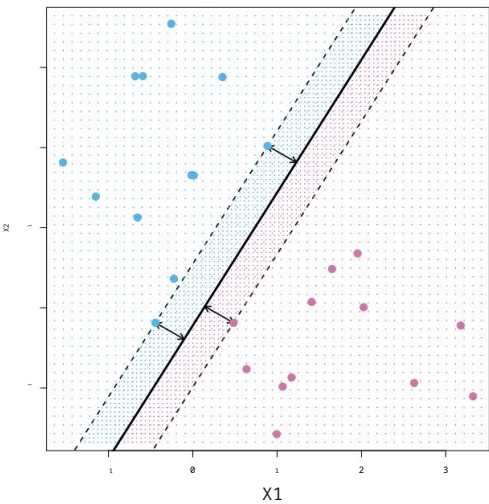
các quan sát tới siêu mặt phẳng, và được gọi là lề. Giá trị tối đa

siêu mặt phẳng biên là siêu mặt phẳng phân tách mà biên độ là biên độ.

lớn nhất-nghĩa là, đó là siêu mặt phẳng có khoảng cách tối thiểu xa nhất đến các quan sát huấn luyện. Sau đó, chúng ta có thể phân loại một quan sát thử nghiệm.

dựa trên vị trí của nó so với siêu mặt phẳng biên độ tối đa. Điều này đã được biết đến.

tối đa
lề
siêu mặt phẳng
tối ưu
tách biệt
siêu mặt phẳng



HÌNH 9.3. Có hai loại quan sát, được hiển thị bằng màu xanh lam và màu trắng. Màu tím. Mặt phẳng biên độ tối đa được hiển thị dưới dạng đường liền nét. Biên độ là khoảng cách từ đường liền nét đến một trong hai đường chấm. Hai đường màu xanh Các điểm và điểm màu tím nằm trên các đường chấm là các vectơ hỗ trợ. và khoảng cách từ những điểm đó đến siêu mặt phẳng được biểu thị bằng các mũi tên. Lưới màu tím và xanh lam biểu thị quy tắc quyết định được đưa ra bởi bộ phân loại dựa trên điều này. siêu mặt phẳng phân tách.

như là bộ phân loại có biên độ tối đa. Chúng tôi hy vọng rằng một bộ phân loại có biên độ lớn trên dữ liệu huấn luyện cũng sẽ có biên độ lớn trên dữ liệu kiểm tra. và do đó sẽ phân loại các quan sát thử nghiệm một cách chính xác. Mặc dù bộ phân loại biên độ tối đa thường thành công, nhưng nó cũng có thể dẫn đến hiện tượng quá khớp khi p rất lớn.

tối đa
lẻ
bộ phân loại

Nếu $\beta_0, \beta_1, \dots, \beta_p$ là các hệ số của siêu mặt phẳng biên độ tối đa, sau đó bộ phân loại biên độ tối đa sẽ phân loại quan sát thử nghiệm x^* dựa trên về dấu của $f(x) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p$.

Hình 9.3 hiển thị siêu mặt phẳng biên độ tối đa trên tập dữ liệu của Hình 9.2. So sánh bảng bên phải của Hình 9.2 với Hình 9.3, Chúng ta thấy rằng siêu mặt phẳng biên độ tối đa được thể hiện trong Hình 9.3 thực sự dẫn đến khoảng cách tối thiểu lớn hơn giữa các quan sát và siêu mặt phẳng phân tách—tức là, một biên độ lớn hơn. Theo một nghĩa nào đó, mức tối đa Siêu mặt phẳng biên đại diện cho đường giữa của "tầm" rộng nhất mà chúng ta có thể Chèn vào giữa hai lớp.

Quan sát Hình 9.3, ta thấy rằng ba quan sát huấn luyện nằm cách đều mặt phẳng biên độ tối đa và nằm dọc theo các đường nét đứt. cho biết độ rộng của lề. Ba quan sát này được gọi là các vectơ hỗ trợ, vì chúng là các vectơ trong không gian p chiều (trong Hình 9.3, $p = 2$) và chúng "hỗ trợ" siêu mặt phẳng biên độ tối đa theo nghĩa đó. Nếu những điểm này được dịch chuyển một chút thì siêu mặt phẳng biên độ tối đa cũng sẽ dịch chuyển theo. Điều thú vị là, siêu mặt phẳng biên độ tối đa phụ thuộc trực tiếp vào các vectơ hỗ trợ, nhưng không phụ thuộc vào các quan sát khác: Việc di chuyển đến bất kỳ vị trí quan sát nào khác sẽ không ảnh hưởng đến quá trình tách biệt. siêu mặt phẳng, với điều kiện là chuyển động của vật quan sát không gây ra điều đó

ứng hộ
vectơ

Vượt qua ranh giới được thiết lập bởi lẽ. Việc siêu mặt phẳng là tối đa phụ thuộc trực tiếp vào chỉ một tập hợp con nhỏ của các quan sát là một thuộc tính quan trọng sẽ xuất hiện sau này trong chương này khi chúng ta thảo luận về bộ phân loại vectơ hỗ trợ và máy vectơ hỗ trợ.

9.1.4 Xây dựng bộ phân loại biên độ tối đa

Bây giờ chúng ta xem xét bài toán xây dựng siêu mặt phẳng biên độ tối đa dựa trên một tập hợp n quan sát huấn luyện $x_1, \dots, x_n \in \mathbb{R}^p$ và các nhãn lớp tương ứng $y_1, \dots, y_n \in \{-1, 1\}$. Nói tóm lại, siêu mặt phẳng biên độ tối đa là lời giải cho bài toán tối ưu hóa.

$$\begin{aligned} & \text{tối đa hóa } M \\ & \beta_0, \beta_1, \dots, \beta_p, M \end{aligned} \quad (9.9)$$

$$\begin{aligned} & \text{tùy thuộc vào } \beta_j = 1, \\ & j=1 \end{aligned} \quad (9.10)$$

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip}) \geq M \quad i=1, \dots, N. \quad (9.11)$$

Bài toán tối ưu hóa này (9.9)-(9.11) thực ra đơn giản hơn về bề ngoài của nó. Trước hết, ràng buộc trong (9.11) là

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip}) \geq M \quad i=1, \dots, n$$

đảm bảo rằng mỗi quan sát sẽ nằm ở phía bên phải của siêu mặt phẳng, với điều kiện M dương. (Thực ra, để mỗi quan sát nằm ở phía bên phải của siêu mặt phẳng, chúng ta chỉ cần $y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip}) > 0$, vì vậy ràng buộc trong (9.11) thực tế yêu cầu mỗi quan sát phải nằm ở phía bên phải của siêu mặt phẳng, với một số khoảng đệm, với điều kiện M dương.)

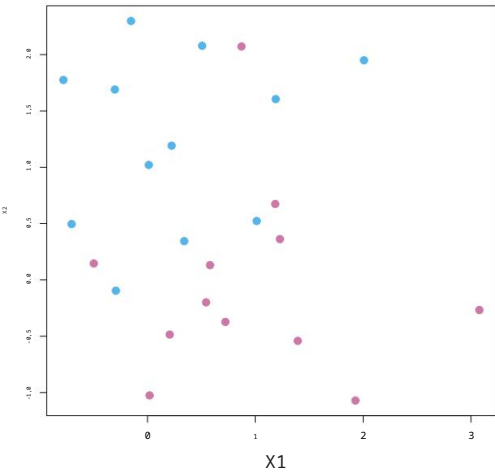
Thứ hai, lưu ý rằng (9.10) thực sự không phải là một ràng buộc đối với siêu mặt phẳng, vì nếu $\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip} = 0$ xác định một siêu mặt phẳng, thì $k(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip}) = 0$ cũng vậy đối với bất kỳ $k \neq 0$ nào. Tuy nhiên, (9.10) bổ sung ý nghĩa cho (9.11); người ta có thể chứng minh rằng với ràng buộc này, khoảng cách vuông góc từ quan sát thứ i đến siêu mặt phẳng được cho bởi

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip}).$$

Do đó, các ràng buộc (9.10) và (9.11) đảm bảo rằng mỗi quan sát nằm ở phía chính xác của siêu mặt phẳng và cách siêu mặt phẳng ít nhất một khoảng cách M . Vì vậy, M biểu thị biên độ của siêu mặt phẳng của chúng ta, và bài toán tối ưu hóa chọn $\beta_0, \beta_1, \dots, \beta_p$ để tối đa hóa M . Đây chính xác là định nghĩa của siêu mặt phẳng biên độ tối đa! Bài toán (9.9)-(9.11) có thể được giải quyết hiệu quả, nhưng chi tiết của quá trình tối ưu hóa này nằm ngoài phạm vi của cuốn sách này.

9.1.5 Trường hợp không thể tách rời

Bộ phân loại biên độ tối đa là một cách rất tự nhiên để thực hiện phân loại, nếu tồn tại một siêu mặt phẳng phân tách. Tuy nhiên, như chúng ta đã gợi ý, trong nhiều trường hợp không tồn tại siêu mặt phẳng phân tách, và do đó không có biên độ tối đa.



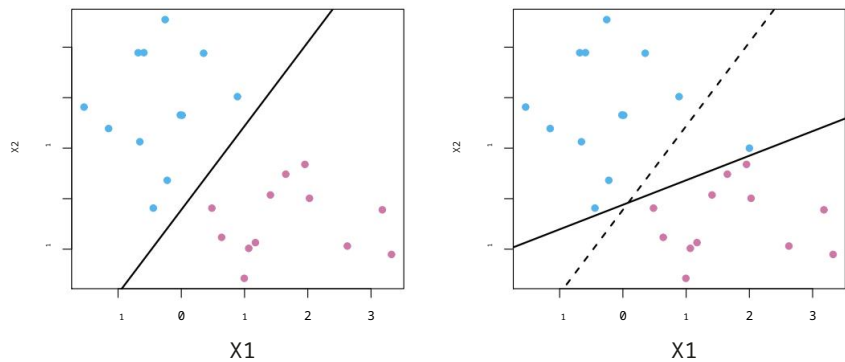
HÌNH 9.4. Có hai loại quan sát, được hiển thị bằng màu xanh lam và màu trắng. màu tím. Trong trường hợp này, hai lớp không thể phân tách bằng một siêu mặt phẳng, và do đó Không thể sử dụng bộ phân loại biên độ tối đa.

bộ phân loại lè. Trong trường hợp này, bài toán tối ưu hóa (9.9)–(9.11) không có lời giải với $M > 0$. Một ví dụ được minh họa trong Hình 9.4. Trong trường hợp này, chúng ta Không thể tách biệt hoàn toàn hai lớp này. Tuy nhiên, như chúng ta sẽ thấy ở phần tiếp theo Trong phần này, chúng ta có thể mở rộng khái niệm về siêu mặt phẳng phân tách để Phát triển một siêu mặt phẳng gần như phân tách các lớp, sử dụng cái gọi là Biên mềm. Sự khái quát hóa của bộ phân loại biên tối đa cho Trường hợp không thể tách rời được gọi là bộ phân loại vectơ hỗ trợ.

9.2 Bộ phân loại vectơ hỗ trợ

9.2.1 Tổng quan về bộ phân loại vectơ hỗ trợ

Trong Hình 9.4, chúng ta thấy rằng các quan sát thuộc hai lớp không phải là nhất thiết phải được phân tách bởi một siêu mặt phẳng. Trên thực tế, ngay cả khi siêu mặt phẳng phân tách tồn tại, thì vẫn có những trường hợp mà bộ phân loại dựa trên Một siêu mặt phẳng phân tách có thể không phải là điều mong muốn. Một bộ phân loại dựa trên một siêu mặt phẳng phân tách nhất thiết sẽ phân loại hoàn hảo tất cả quá trình huấn luyện. các quan sát; điều này có thể dẫn đến sự nhạy cảm đối với các quan sát riêng lẻ. Một ví dụ được minh họa trong Hình 9.5. Việc thêm một quan sát duy nhất vào Hình 9.5, phần bên phải, dẫn đến sự thay đổi đáng kể trong siêu mặt phẳng biên độ tối đa. Siêu mặt phẳng biên độ tối đa thu được không phải là Tạm ổn – thứ nhất, nó chỉ có một biên độ rất nhỏ. Điều này gây ra vấn đề. vì như đã thảo luận trước đó, khoảng cách của một điểm quan sát từ Siêu mặt phẳng có thể được xem như một thước đo mức độ tin cậy của chúng ta rằng quan sát đã được phân loại chính xác. Hơn nữa, thực tế là siêu mặt phẳng biên độ tối đa cực kỳ nhạy cảm với sự thay đổi trong một quan sát duy nhất. Điều này cho thấy có thể mô hình đã bị quá khớp với dữ liệu huấn luyện. Trong trường hợp này, chúng ta có thể sẵn sàng xem xét một bộ phân loại dựa trên siêu mặt phẳng không hoàn toàn phân tách hai lớp, vì lợi ích của việc...



HÌNH 9.5. Bên trái: Hai loại quan sát được hiển thị bằng màu xanh lam và màu trắng. màu tím, cùng với siêu mặt phẳng biên độ tối đa. Bên phải: Một màu xanh lam bổ sung đã bổ sung thêm dữ liệu quan sát, dẫn đến sự thay đổi đáng kể về biên độ tối đa. Siêu mặt phẳng được thể hiện bằng đường liền nét. Đường nét đứt biểu thị biên độ tối đa. siêu mặt phẳng thu được khi không có điểm bổ sung này.

- Độ bền vững cao hơn đối với các quan sát riêng lẻ, và
- Phân loại tốt hơn hầu hết các quan sát trong quá trình huấn luyện.

Tức là, việc phân loại sai một vài dữ liệu huấn luyện đôi khi lại có thể mang lại lợi ích. để có thể phân loại các quan sát còn lại một cách tốt hơn.

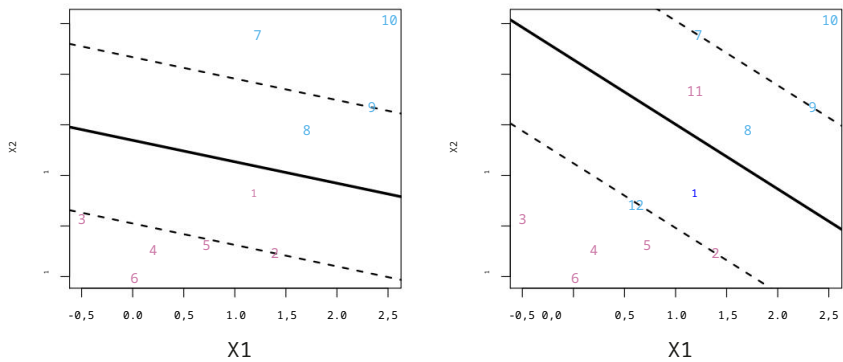
Bộ phân loại vectơ hỗ trợ, đôi khi được gọi là bộ phân loại biên mềm, thực hiện chính xác điều này. Thay vì tìm kiếm biên lớn nhất có thể để... mọi quan sát không chỉ nằm ở phía bên phải của siêu mặt phẳng mà còn cũng ở phía bên phải của lề, thay vào đó chúng ta cho phép một số quan sát. nằm ở phía sai lề, hoặc thậm chí là phía sai của... siêu mặt phẳng. (Giới hạn này không cố định vì nó có thể bị vi phạm bởi một số yếu tố (của các quan sát huấn luyện.) Một ví dụ được hiển thị ở bảng bên trái. Hình 9.6. Hầu hết các quan sát đều nằm ở phía bên phải của lề. Tuy nhiên, một phần nhỏ các quan sát lại nằm ở phía sai của phạm vi. lề.

ứng hộ
vectơ
bộ phân loại
biên độ mềm
bộ phân loại

Một nhận xét không chỉ có thể nằm ở phía sai của lề mà còn ở phía sai của siêu mặt phẳng. Trên thực tế, khi không có sự phân tách siêu mặt phẳng, tình huống như vậy là không thể tránh khỏi. Quan sát ở phía sai của Siêu mặt phẳng tương ứng với các quan sát huấn luyện bị phân loại sai bởi bộ phân loại vectơ hỗ trợ. Hình bên phải của Hình 9.6 minh họa điều này. một kịch bản như vậy.

9.2.2 Chi tiết về Bộ phân loại vectơ hỗ trợ

Bộ phân loại vectơ hỗ trợ phân loại một quan sát thử nghiệm dựa trên... Nó nằm ở phía nào của siêu mặt phẳng. Siêu mặt phẳng được chọn để chính xác phân chia hầu hết các quan sát huấn luyện thành hai lớp, nhưng có thể



HÌNH 9.6. Bên trái: Một bộ phân loại vectơ hỗ trợ được huấn luyện trên một tập dữ liệu nhỏ. Siêu mặt phẳng được biểu thị bằng đường liền nét, còn lề được biểu thị bằng đường nét đứt. Các quan sát màu tím: Các quan sát 3, 4, 5 và 6 nằm ở phía bên phải của Lề, quan sát 2 nằm trên lề, và quan sát 1 nằm ở phía bên kia của lề. Các quan sát màu xanh: Quan sát 7 và 10 nằm ở phía bên phải của Lề, quan sát 9 nằm trên lề, và quan sát 8 nằm ở phía sai của lề. Không có quan sát nào nằm ở phía sai của siêu mặt phẳng. Phải: Tương tự như bảng bên trái với hai điểm bổ sung, 11 và 12. Hai quan sát này nằm ở phía sai của siêu mặt phẳng và phía sai của lề.

Phân loại sai một vài quan sát. Đó là giải pháp cho bài toán tối ưu hóa.

$$\begin{aligned} & \text{tối đa hóa} && M \\ & \beta_0, \beta_1, \dots, \beta_p, 1, \dots, n, M \\ & P \end{aligned} \tag{9.12}$$

$$\begin{aligned} & \text{tùy thuộc vào} && \beta_j = 1, \\ & j=1 \end{aligned} \tag{9.13}$$

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip}) \geq M(1 - i), \tag{9.14}$$

$$\begin{aligned} & i \geq 0, && i \leq C, \\ & i=1 \end{aligned} \tag{9.15}$$

trong đó C là tham số điều chỉnh không âm. Như trong (9.11), M là độ rộng của lề; chúng tôi tìm cách làm cho số lượng này càng lớn càng tốt. Trong (9.14), 1, ..., n là các biến phụ cho phép các quan sát riêng lẻ nằm trong phạm vi cho phép. Phía sai của lề hoặc siêu mặt phẳng; chúng ta sẽ giải thích chúng trong phần sau. Chỉ tiết hơn sẽ được trình bày ngay sau đây. Sau khi chúng ta giải quyết xong (9.12)–(9.15), chúng ta sẽ phân loại một quan sát thử nghiệm x^* như trước, bằng cách đơn giản xác định ở phía nào của Nó nằm trên siêu mặt phẳng. Tức là, chúng ta phân loại quan sát thử nghiệm dựa trên... dấu của $f(x^*) = \beta_0 + \beta_1 x^*_1 + \dots + \beta_p x^*_p$.

Vấn đề (9.12)–(9.15) có vẻ phức tạp, nhưng cần hiểu rõ hơn về hành vi của nó. Có thể rút ra kết luận thông qua một loạt các quan sát đơn giản được trình bày dưới đây. Đầu tiên. Tương đối so với siêu mặt phẳng và tương đối so với lề. Nếu $\beta_0 = 0$ thì thứ i... Quan sát nằm ở phía bên phải của lề, như chúng ta đã thấy trong Phần 9.1.4. Nếu $i > 0$ thì quan sát thứ i nằm ở phía sai của lề, và Chúng ta nói rằng quan sát thứ i đã vi phạm giới hạn. Nếu $i > 1$ thì đó là ở phía sai của siêu mặt phẳng.

tự lỏng lẻo
biến

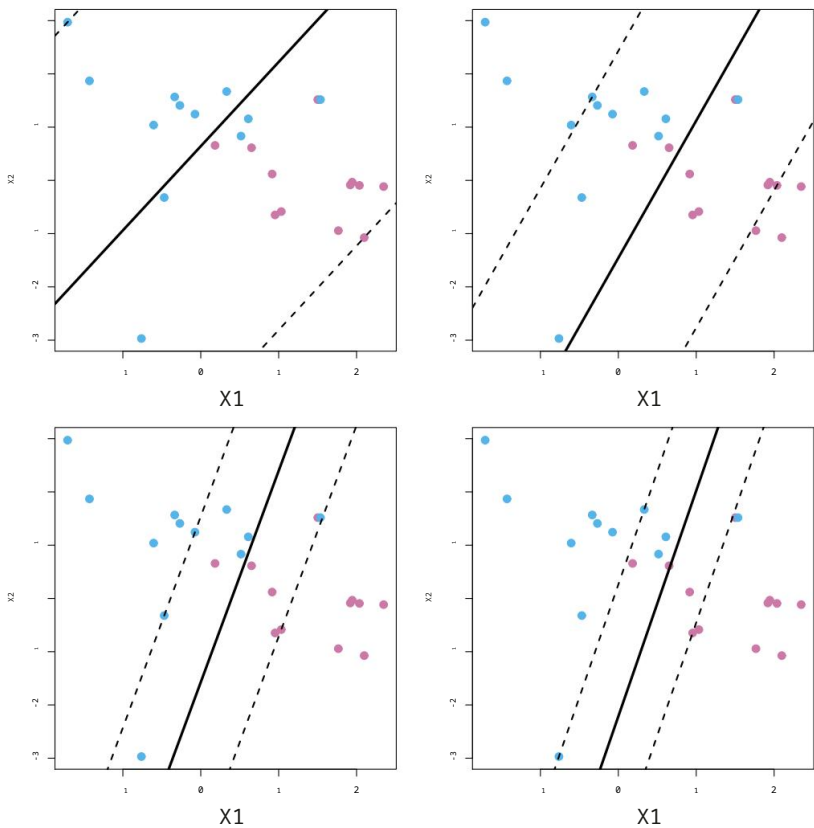
Bây giờ chúng ta xem xét vai trò của tham số điều chỉnh C . Trong (9.15), C giới hạn Tổng của các i , và do đó nó xác định số lượng và mức độ nghiêm trọng của các vi phạm đối với lề (và đối với siêu mặt phẳng) mà chúng ta sẽ chấp nhận. Chúng ta có thể Hãy coi C như một ngân sách cho số tiền mà biên độ an toàn có thể bị vi phạm. bởi n quan sát. Nếu $C = 0$ thì không có ngân sách cho các vi phạm. lề, và phải là trường hợp $n = 0$, trong trường hợp đó ... = (9.12)-(9.15) đơn giản chỉ là bài toán tối ưu hóa siêu mặt phẳng biên độ tối đa (9.9)-(9.11). (Tất nhiên, một siêu mặt phẳng biên độ tối đa tồn tại) (chỉ khi hai lớp có thể tách biệt.) Đối với $C > 0$, không quá C quan sát có thể nằm ở phía sai của siêu mặt phẳng, bởi vì nếu một quan sát nếu nằm ở phía sai của siêu mặt phẳng thì $i=1 \leq C$. Khi $i > 1$ và (9.15) yêu cầu ngân sách C tăng lên, chúng ta trở nên khoan dung hơn với vi phạm lề, và do đó lề sẽ mở rộng. Ngược lại, khi C Khi mức độ giảm đi, chúng ta trở nên ít khoan dung hơn đối với những vi phạm giới hạn và do đó... Lề thu hẹp lại. Một ví dụ được minh họa trong Hình 9.7.

Trên thực tế, C được coi là một tham số điều chỉnh thường được lựa chọn thông qua Kiểm định chéo. Cũng như các tham số điều chỉnh mà chúng ta đã thấy xuyên suốt cuốn sách này, C kiểm soát sự đánh đổi giữa độ lệch và phương sai của kỹ thuật học thống kê. Khi C nhỏ, chúng ta tìm kiếm các biên độ hẹp hiếm khi bị vi phạm; điều này tương đương với một bộ phân loại có độ phù hợp cao với dữ liệu, điều này có thể có độ lệch thấp nhưng phương sai cao. Mặt khác, khi C lớn hơn, Khoảng cách an toàn rộng hơn và chúng ta cho phép nhiều vi phạm hơn; điều này tương đương với việc giúp việc khớp dữ liệu dễ dàng hơn và thu được một bộ phân loại có tiềm năng hơn. Có thể bị thiên lệch nhưng có phương sai thấp hơn.

Bài toán tối ưu hóa (9.12)-(9.15) có một đặc tính rất thú vị: hóa ra chỉ những quan sát nằm ở rìa hoặc những quan sát đó mới có giá trị. Vi phạm lề sẽ ảnh hưởng đến siêu mặt phẳng, và do đó ảnh hưởng đến bộ phân loại thu được. Nói cách khác, một quan sát nằm hoàn toàn ở phía bên phải. Việc thay đổi lề không ảnh hưởng đến bộ phân loại vectơ hỗ trợ! Việc thay đổi vị trí của quan sát đó sẽ không làm thay đổi bộ phân loại chút nào, miễn là Vị trí của nó vẫn nằm ở phía bên phải của lề. Quan sát nằm ngay trên lề hoặc nằm sai phía lề. Lớp của chúng được gọi là vectơ hỗ trợ. Những quan sát này có ảnh hưởng đến... Bộ phân loại vectơ hỗ trợ.

Việc chỉ có các vectơ hỗ trợ ảnh hưởng đến bộ phân loại là phù hợp với quan điểm của chúng tôi. Khẳng định trước đó cho rằng C kiểm soát sự đánh đổi giữa độ lệch và phương sai của vùng hỗ trợ bộ phân loại vectơ. Khi tham số điều chỉnh C lớn, thì biên độ là Rộng, nhiều quan sát vi phạm giới hạn, và do đó có nhiều bằng chứng hỗ trợ vectơ. Trong trường hợp này, nhiều quan sát liên quan đến việc xác định siêu mặt phẳng. Hình 9.7 minh họa bằng phía trên bên trái cho thấy thiết lập này: Bộ phân loại có phương sai thấp (vì nhiều quan sát là vectơ hỗ trợ) nhưng có khả năng sai lệch cao. Ngược lại, nếu C nhỏ thì sẽ có ít hơn. Các vectơ hỗ trợ và do đó bộ phân loại thu được sẽ có độ lệch thấp. Độ biến thiên cao. Hình 9.7 minh họa thiết lập này ở góc dưới bên phải. Chỉ với tám vectơ hỗ trợ.

Thực tế là quy tắc quyết định của bộ phân loại vectơ hỗ trợ chỉ dựa trên... Trên một tập hợp con nhỏ tiềm năng của các quan sát huấn luyện (các vectơ hỗ trợ) có nghĩa là nó khá mạnh mẽ đối với hành vi của các quan sát đó. nằm rất xa mặt phẳng siêu phẳng. Đặc tính này khác biệt so với một số đặc tính khác.

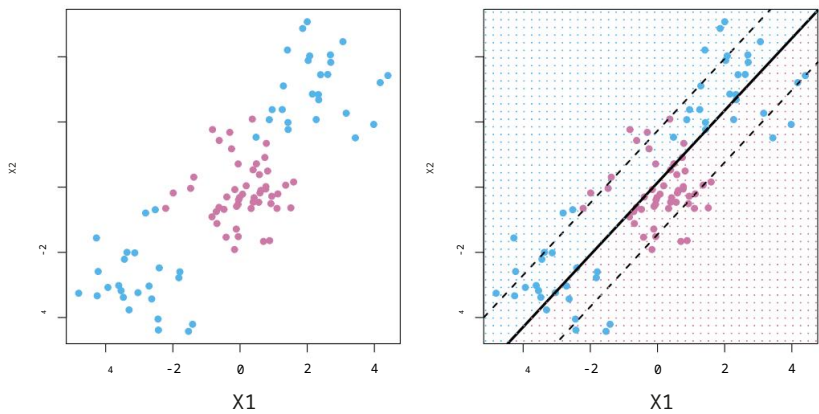


HÌNH 9.7. Một bộ phân loại vectơ hỗ trợ được huấn luyện bằng cách sử dụng bốn giá trị khác nhau của tham số điều chỉnh C trong (9.12)–(9.15). Giá trị lớn nhất của C đã được sử dụng ở bảng phía trên bên trái, các giá trị nhỏ hơn được sử dụng, còn ở bảng phía trên bên phải và bảng phía dưới bên trái, và các tấm phía dưới bên phải. Khi C lớn, thì sẽ có dung sai cao cho các quan sát nằm ở phía sai của lề, và do đó lề sẽ là lớn. Khi C giảm, dung sai cho các quan sát nằm ngoài phạm vi cho phép. Biên độ giảm dần, và biên độ thu hẹp lại.

các phương pháp phân loại khác mà chúng ta đã thấy trong các chương trước, chẳng hạn như phân tích phân biệt tuyến tính. Hãy nhớ rằng quy tắc phân loại LDA phụ thuộc vào giá trị trung bình của tất cả các quan sát trong mỗi lớp, cũng như Ma trận hiệp phương sai trong lớp được tính toán bằng cách sử dụng tất cả các quan sát. Ngược lại, không giống như LDA, hồi quy logistic có độ nhạy rất thấp đối với các quan sát nằm xa ranh giới quyết định. Trên thực tế, chúng ta sẽ thấy điều này trong Phần 9.5. Điều đó cho thấy bộ phân loại vectơ hỗ trợ và hồi quy logistic có mối liên hệ mật thiết với nhau.

9.3 Máy hỗ trợ vectơ

Trước tiên, chúng ta sẽ thảo luận về một cơ chế tổng quát để chuyển đổi bộ phân loại tuyến tính thành... một phương pháp tạo ra các ranh giới quyết định phi tuyến tính. Sau đó, chúng tôi giới thiệu... Máy hỗ trợ vectơ (SVM) thực hiện việc này một cách tự động.



HÌNH 9.8. Bên trái: Các quan sát được chia thành hai lớp, với ranh giới không tuyến tính giữa chúng. Bên phải: Bộ phân loại vectơ hỗ trợ tìm kiếm một ranh giới tuyến tính, ranh giới, và do đó hoạt động rất kém.

9.3.1 Phân loại với ranh giới quyết định phi tuyến tính

Bộ phân loại vectơ hỗ trợ là một phương pháp tự nhiên để phân loại trong... Đây là trường hợp phân chia thành hai lớp nếu ranh giới giữa hai lớp là tuyến tính. Tuy nhiên, trên thực tế, đôi khi chúng ta gặp phải các ranh giới lớp không tuyến tính. Ví dụ, hãy xem xét dữ liệu ở bảng bên trái của Hình 9.8. Đó là rõ ràng rằng bộ phân loại vectơ hỗ trợ hoặc bất kỳ bộ phân loại tuyến tính nào sẽ hoạt động ở đây thì không tốt lắm. Thật vậy, bộ phân loại vectơ hỗ trợ được hiển thị ở bên phải.

Phần hình minh họa trong Hình 9.8 không hữu ích ở đây.

Trong Chương 7, chúng ta sẽ gặp một tình huống tương tự. Chúng ta thấy ở đó Hiệu suất của hồi quy tuyến tính có thể bị ảnh hưởng khi có mối quan hệ phi tuyến tính giữa các biến dự đoán và biến kết quả. Trong trường hợp đó, Chúng tôi xem xét việc mở rộng không gian đặc trưng bằng cách sử dụng các hàm của các biến dự đoán. Chẳng hạn như các số hạng bậc hai và bậc ba, nhằm giải quyết tính phi tuyến tính này. Trong trường hợp bộ phân loại vectơ hỗ trợ, chúng ta có thể giải quyết vấn đề về ranh giới phi tuyến tính giữa các lớp theo cách tương tự, bằng cách Mở rộng không gian đặc trưng bằng cách sử dụng các phương trình bậc hai, bậc ba, và thậm chí bậc cao hơn. các hàm đa thức của các biến dự đoán. Ví dụ, thay vì khớp một Bộ phân loại vectơ hỗ trợ sử dụng đặc trưng p

$$X_1, X_2, \dots, X_p,$$

Thay vào đó, chúng ta có thể sử dụng bộ phân loại vectơ hỗ trợ với các đặc trưng $2p$.

$$X_1, X_2, \dots, X_p, X_1^2, X_2^2, \dots, X_p^2, \dots$$

Khi đó (9.12)-(9.15) sẽ trở thành

tối đa hóa

$$\beta_0, \beta_{11}, \beta_{12}, \dots, \beta_{p1}, \beta_{p2}, 1, \dots, n, M$$

(9.16)

tùy thuộc vào yi

$$\beta_0 + \sum_{j=1}^p \beta_{j1} x_{ij} + \sum_{j=1}^p \beta_{j2} x_{2j} \geq M(1 - i),$$

$$\beta_0 + \sum_{j=1}^p \beta_{j1} x_{ij} + \sum_{j=1}^p \beta_{j2} x_{2j} \leq M(1 + i),$$

$$\beta_0 + \sum_{j=1}^p \beta_{j1} x_{ij} + \sum_{j=1}^p \beta_{j2} x_{2j} = 1.$$

Tại sao điều này lại dẫn đến ranh giới quyết định phi tuyến tính? Trong hình phóng to không gian đặc trưng, ranh giới quyết định thu được từ (9.16) thực tế là tuyến tính. Nhưng trong không gian đặc trưng ban đầu, ranh giới quyết định có dạng $q(x)=0$, trong đó q là một đa thức bậc hai, và các nghiệm của nó nói chung là phi tuyến tính. Ngoài ra, người ta cũng có thể muốn mở rộng không gian đặc trưng. với các số hạng đa thức bậc cao hơn, hoặc với các số hạng tương tác có dạng $X_j X_k$ cho $j \neq k$. Ngoài ra, các hàm khác của các biến dự đoán cũng có thể được sử dụng. nên được xem xét thay vì các đa thức. Không khó để thấy rằng có rất nhiều cách khả thi để mở rộng không gian đặc trưng, và trừ khi chúng ta Nếu không cẩn thận, chúng ta có thể sẽ có một số lượng lớn các đặc trưng. Khi đó, việc tính toán sẽ trở nên khó quản lý. Máy vectơ hỗ trợ (SVM) là một ví dụ. Phần trình bày tiếp theo cho phép chúng ta mở rộng không gian tính năng được hỗ trợ. Bộ phân loại vectơ theo cách dẫn đến tính toán hiệu quả.

9.3.2 Máy hỗ trợ vectơ

Máy vectơ hỗ trợ (SVM) là một phần mở rộng của bộ phân loại vectơ hỗ trợ, được tạo ra bằng cách mở rộng không gian đặc trưng theo một cách cụ thể. sử dụng nhân. Bây giờ chúng ta sẽ thảo luận về phần mở rộng này, chi tiết như sau: máy móc Vấn đề này khá phức tạp và nằm ngoài phạm vi của cuốn sách này. Tuy nhiên, cốt lõi chính vẫn là vấn đề chính. Ý tưởng này được mô tả trong Mục 9.3.1: chúng ta có thể muốn mở rộng không gian tính năng của mình. nhằm mục đích tạo ra ranh giới phi tuyến tính giữa các lớp. Phương pháp tiếp cận dựa trên nhân mà chúng tôi mô tả ở đây chỉ đơn giản là một phương pháp tính toán hiệu quả. cách tiếp cận để hiện thực hóa ý tưởng này.

Chúng ta chưa thảo luận chi tiết về cách tính toán bộ phân loại vectơ hỗ trợ vì các chi tiết này khá phức tạp về mặt kỹ thuật. Tuy nhiên, nó... ra rằng giải pháp cho vấn đề phân loại vectơ hỗ trợ (9.12)-(9.15) chỉ liên quan đến tích vô hướng của các quan sát (trái ngược với (các quan sát tự thân). Tích vô hướng của hai vectơ x chiều a và b là được định nghĩa là $a \cdot b = \sum_{i=1}^a b_i b_i$. Do đó, tích vô hướng của hai quan sát x_i, x_j được cho bởi

$$x_i \cdot x_j = \sum_{j=1}^p x_{ij} x_{ij}.$$

(9.17)

Có thể chứng minh rằng

- Bộ phân loại vectơ hỗ trợ tuyến tính có thể được biểu diễn như sau:

$$f(x) = \beta_0 + \sum_{i=1}^n a_i x_i, x_i,$$

(9.18)

trong đó có n tham số $a_i, i = 1, \dots, n$, mỗi tham số tương ứng với một quan sát trong quá trình huấn luyện.

- Để ước tính các tham số $a_1, \dots, a_n$ và β_0 , tất cả những gì chúng ta cần là tích vô hướng x_i, x_i giữa tất cả các cặp quan sát huấn luyện. (Ký hiệu này có nghĩa là $n(n-1)/2$, và cho biết số cặp trong một tập hợp gồm n mục.)

Lưu ý rằng trong (9.18), để đánh giá hàm $f(x)$, chúng ta cần tính tích vô hướng giữa điểm mới x và mỗi điểm huấn luyện x_i . Tuy nhiên, hóa ra a_i chỉ khác 0 đối với các vectơ hỗ trợ trong lời giải – nghĩa là, nếu một quan sát huấn luyện không phải là vectơ hỗ trợ, thì a_i của nó bằng 0. Vì vậy, nếu S là tập hợp các chỉ số của các điểm hỗ trợ này, chúng ta có thể viết lại bất kỳ hàm lời giải nào có dạng (9.18) như sau:

$$f(x) = \beta_0 + \sum_{i \in S} a_i x_i, \tag{9.19}$$

Điều này thường liên quan đến ít thuật ngữ hơn nhiều so với trong

(9.18).² Tóm lại, trong việc biểu diễn bộ phân loại tuyến tính $f(x)$ và trong việc tính toán các hệ số của nó, tất cả những gì chúng ta cần là các tích vô hướng.

Bây giờ giả sử rằng mỗi khi tích vô hướng (9.17) xuất hiện trong biểu diễn (9.18) hoặc trong phép tính giải pháp cho bộ phân loại vectơ hỗ trợ, chúng ta thay thế nó bằng một dạng tổng quát của tích vô hướng có dạng

$$K(x_i, x_j), \tag{9.20}$$

trong đó K là một hàm nào đó mà chúng ta sẽ gọi là hạt nhân. Hạt nhân là một hàm định lượng sự tương đồng giữa hai quan sát. Ví dụ, chúng ta có thể đơn giản lấy

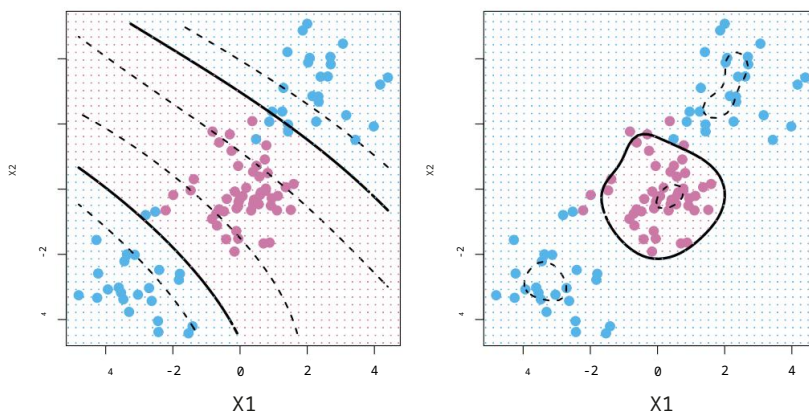
$$K(x_i, x_j) = \sum_{p=1}^P x_{ij} x_{jp}, \tag{9.21}$$

Điều này sẽ chỉ trả lại cho chúng ta bộ phân loại vectơ hỗ trợ. Phương trình 9.21 được gọi là hạt nhân tuyến tính vì bộ phân loại vectơ hỗ trợ là tuyến tính đối với các đặc trưng; hạt nhân tuyến tính về cơ bản định lượng sự tương đồng của một cặp quan sát bằng cách sử dụng hệ số tương quan Pearson (chuẩn). Nhưng thay vào đó, người ta có thể chọn một dạng khác cho (9.20). Ví dụ, người ta có thể thay thế mọi trường hợp của $\sum_{j=1}^P x_{ij} x_{jp}$ với số lượng

$$K(x_i, x_j) = (1 + \sum_{j=1}^P x_{ij} x_{jp})^d. \tag{9.22}$$

Đây được gọi là hạt nhân đa thức bậc d , trong đó d là một số nguyên dương. Việc sử dụng hạt nhân như vậy với $d > 1$, thay vì hạt nhân tuyến tính tiêu chuẩn (9.21), trong thuật toán phân loại vectơ hỗ trợ dẫn đến ranh giới quyết định linh hoạt hơn nhiều. Về cơ bản, nó tương đương với việc khớp một vectơ hỗ trợ

²Bằng cách khai triển từng tích vô hướng trong (9.19), dễ dàng thấy rằng $f(x)$ là một hàm tuyến tính của tọa độ x . Việc này cũng thiết lập sự tương ứng giữa a_i và các tham số ban đầu β_j .



HÌNH 9.9. Bên trái: Một SVM với hạt nhân đa thức bậc 3 được áp dụng cho dữ liệu phi tuyến tính từ Hình 9.8 dẫn đến một quyết định phù hợp hơn nhiều. Bên phải: Một SVM với hạt nhân xuyên tâm được áp dụng. Trong ví dụ này, một trong hai hạt nhân được sử dụng, có khả năng nắm bắt ranh giới quyết định.

bộ phân loại trong không gian đa chiều liên quan đến đa thức bậc d , thay vì trong không gian đặc trưng ban đầu. Khi bộ phân loại vectơ hỗ trợ được kết hợp với một hạt nhân phi tuyến tính như (9.22), bộ phân loại kết quả là được gọi là máy vectơ hỗ trợ. Lưu ý rằng trong trường hợp này (phi tuyến tính) hàm có dạng

$$f(x) = \beta_0 + \sum_{i=1}^S \alpha_i K(x, x_i). \quad (9.23)$$

Hình 9.9, bảng bên trái, hiển thị một ví dụ về SVM với...

Hạt nhân đa thức được áp dụng cho dữ liệu phi tuyến tính từ Hình 9.8. Kết quả phù hợp là: một sự cải tiến đáng kể so với bộ phân loại vectơ hỗ trợ tuyến tính. Khi $d = 1$, thì SVM sẽ trở thành bộ phân loại vectơ hỗ trợ đã thấy trước đó trong chương này.

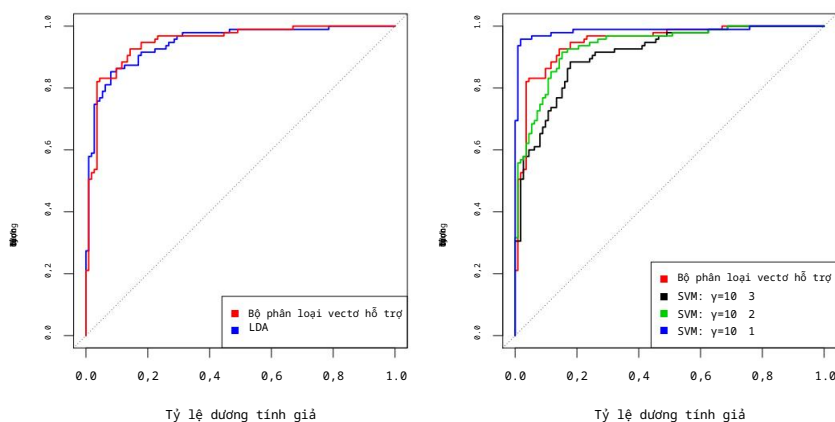
Hạt nhân đa thức được hiển thị trong (9.22) là một ví dụ về một khả năng có thể hạt nhân phi tuyến tính, nhưng có rất nhiều lựa chọn thay thế. Một lựa chọn phổ biến khác là... hạt nhân xuyên tâm, có dạng

$$K(x_i, x_j) = \exp\left(-\gamma \sum_{j=1}^P (x_{ij} - x_{ij})^2\right). \quad (9.24)$$

Trong (9.24), γ là một hằng số dương. Bảng bên phải của Hình 9.9 cho thấy một ví dụ về SVM với hạt nhân xuyên tâm trên dữ liệu phi tuyến tính này; nó cũng làm rất tốt việc phân tách hai nhóm.

Hạt nhân xuyên tâm (9.24) thực sự hoạt động như thế nào? Nếu một quan sát thử nghiệm $x = (x_1, \dots, x_P)^T$ khác xa so với quan sát huấn luyện x_i về mặt khoảng cách Euclidean, thì $\sum_{j=1}^P (x_{ij} - x_j)^2$ sẽ lớn, và do đó $K(x, x_i) = \exp\left(-\gamma \sum_{j=1}^P (x_{ij} - x_j)^2\right)$ sẽ rất nhỏ. Điều này có nghĩa là trong (9.23), x_i sẽ gần như không đóng vai trò gì trong $f(x)$. Nhớ lại rằng nhãn lớp dự đoán cho Quan sát thử nghiệm x dựa trên dấu của $f(x)$. Nói cách khác, huấn luyện Các quan sát nằm xa x^* về cơ bản sẽ không đóng vai trò gì trong việc dự đoán nhãn lớp cho x^* . Điều này có nghĩa là hạt nhân xuyên tâm có tính cục bộ rất cao.

hạt nhân xuyên tâm



HÌNH 9.10. Đường cong ROC cho tập dữ liệu huấn luyện **Heart**. Bên trái: Vùng hỗ trợ So sánh bộ phân loại vectơ hỗ trợ và LDA. Bên phải: Bộ phân loại vectơ hỗ trợ so sánh với SVM sử dụng hạt nhân cơ sở tuyến tâm với $\gamma = 10^{-3}$, 10^{-2} và 10^{-1} .

hành vi, theo nghĩa là chỉ những quan sát huấn luyện gần đó mới có tác dụng trên nhãn lớp của một quan sát thử nghiệm.

Việc sử dụng kernel có ưu điểm gì hơn so với việc chỉ đơn giản là phóng to?

không gian đặc trưng sử dụng các hàm của các đặc trưng ban đầu, như trong (9.16)? Một

Ưu điểm nằm ở khả năng tính toán, và điều đó thể hiện ở việc sử dụng các nhân (kernels),

Người ta chỉ cần tính $K(x_i, x_j)$ cho tất cả các cặp i, j khác nhau. Điều này có thể là được thực hiện mà không cần làm việc rõ ràng trong không gian đặc trưng mở rộng. Điều này rất quan trọng vì trong nhiều ứng dụng của SVM, không gian đặc trưng mở rộng

Nó quá lớn đến mức việc tính toán trở nên bất khả thi. Đối với một số nhân, chẳng hạn như...

hạt nhân tuyến tâm (9.24), không gian đặc trưng là ngậm định và vô hạn chiều,

Vì vậy, dù sao thì chúng ta cũng không thể thực hiện các phép tính ở đó được!

9.3.3 Ứng dụng vào dữ liệu bệnh tim mạch

Trong Chương 8, chúng ta sẽ áp dụng cây quyết định và các phương pháp liên quan vào dữ liệu về tim.

Mục tiêu là sử dụng 13 biến dự báo như Tuổi, Giới tính và Cholesterol để dự đoán.

Liệu một cá nhân có mắc bệnh tim hay không. Bây giờ chúng ta sẽ nghiên cứu cách thức hoạt động của SVM. so sánh với LDA trên tập dữ liệu này. Sau khi loại bỏ 6 quan sát bị thiếu, thì

Dữ liệu bao gồm 297 đối tượng, được chúng tôi chia ngẫu nhiên thành 207 đối tượng huấn luyện và 90 đối tượng thử nghiệm. 90 quan sát thử nghiệm.

Đầu tiên, chúng tôi áp dụng thuật toán LDA và bộ phân loại vectơ hỗ trợ vào dữ liệu huấn luyện.

Lưu ý rằng bộ phân loại vectơ hỗ trợ tương đương với một SVM sử dụng hạt nhân đa thức bậc $d = 1$. Bảng bên trái của Hình 9.10 hiển thị

Đường cong ROC (được mô tả trong Phần 4.4.2) cho các dự đoán của tập huấn luyện.

Cả LDA và bộ phân loại vectơ hỗ trợ đều tính toán điểm số. Cả hai bộ phân loại đều tính điểm.

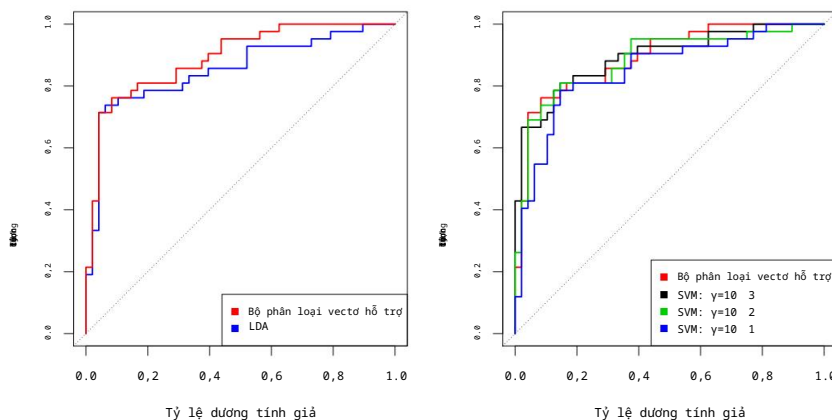
có dạng $\hat{f}(X) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p$ cho mỗi quan sát.

Với bất kỳ ngưỡng t nào, chúng ta phân loại các quan sát thành nhóm bệnh tim hoặc nhóm bệnh tim.

không có phân loại bệnh tim nào tùy thuộc vào việc $\hat{f}(X) < t$ hay $\hat{f}(X) \geq t$.

Đường cong ROC được thu được bằng cách hình thành các dự đoán này và tính toán.

Tỷ lệ dương tính giả và dương tính thật cho một loạt các giá trị của t . Một bộ phân loại tối ưu sẽ nằm sát góc trên bên trái của biểu đồ ROC. Trong trường hợp này



HÌNH 9.11. Đường cong ROC cho tập dữ liệu thử nghiệm của dữ liệu *Tim*. Bên trái: Vùng hồ trợ So sánh bộ phân loại vectơ hỗ trợ và LDA. Bên phải: Bộ phân loại vectơ hỗ trợ là so sánh với SVM sử dụng hạt nhân cơ sở tuyến tính với $\gamma = 10^{-3}$, 10^{-2} và 10^{-1} .

Cả LDA và bộ phân loại vectơ hỗ trợ đều hoạt động tốt, mặc dù có một số hạn chế.

Gợi ý rằng bộ phân loại vectơ hỗ trợ có thể nhỉnh hơn một chút.

Bảng bên phải của Hình 9.10 hiển thị các đường cong ROC cho SVM sử dụng một hạt nhân tuyến tính, với các giá trị khác nhau của γ . Khi γ tăng lên và sự phù hợp trở nên Đường cong ROC càng phi tuyến tính thì càng được cải thiện. Sử dụng $\gamma = 10^{-1}$ dường như mang lại kết quả như mong muốn. Đường cong ROC gần như hoàn hảo. Tuy nhiên, những đường cong này thể hiện quá trình huấn luyện tỷ lệ lỗi, có thể gây hiểu nhầm về hiệu suất trong các bài kiểm tra mới dữ liệu. Hình 9.11 hiển thị các đường cong ROC được tính toán trên 90 quan sát thử nghiệm. Chúng ta nhận thấy một số khác biệt so với các đường cong ROC huấn luyện. Trong Ở bảng bên trái của Hình 9.11, bộ phân loại vectơ hỗ trợ dường như có một lợi thế nhỏ so với LDA (mặc dù sự khác biệt này không có ý nghĩa thống kê). Ở bảng bên phải, SVM sử dụng $\gamma = 10^{-1}$, mà Cho kết quả tốt nhất trên dữ liệu huấn luyện, nhưng lại đưa ra ước tính tệ nhất trên dữ liệu thử nghiệm. Điều này một lần nữa chứng minh rằng trong khi một phương pháp linh hoạt hơn thì... Phương pháp này thường tạo ra tỷ lệ lỗi huấn luyện thấp hơn, nhưng điều này không nhất thiết dẫn đến hiệu suất được cải thiện trên dữ liệu kiểm thử. Các SVM với $\gamma = 10^{-2}$ và $\gamma = 10^{-3}$ cho hiệu suất tương đương với bộ phân loại vectơ hỗ trợ, và tất cả Ba phương pháp này hoạt động tốt hơn SVM với $\gamma = 10^{-1}$.

9.4 SVM với nhiều hơn hai lớp

Cho đến nay, cuộc thảo luận của chúng ta mới chỉ giới hạn ở trường hợp phân loại nhị phân:

Tức là, phân loại trong môi trường hai lớp. Làm thế nào chúng ta có thể mở rộng SVM?

Còn trường hợp tổng quát hơn, khi chúng ta có một số lượng lớp bất kỳ thì sao?

Hóa ra, khái niệm về các siêu mặt phẳng phân tách mà trên đó SVM hoạt động dựa trên cơ sở đó, nó không tự nhiên phù hợp với nhiều hơn hai lớp. Mặc dù

Một số đề xuất mở rộng SVM sang trường hợp lớp K đã được đưa ra.

Trong số đó, hai thể loại phổ biến nhất là đầu một chọi một và đầu một chọi tất cả.

các phương pháp tiếp cận. Chúng ta sẽ thảo luận ngắn gọn về hai phương pháp tiếp cận đó ở đây.

9.4.1 Phân loại một đối một

Giả sử chúng ta muốn thực hiện phân loại bằng SVM, và có $K > 2$ lớp. Phương pháp so sánh từng cặp (one-versus-one hoặc all-pairs) xây dựng các SVM, mỗi SVM so sánh một cặp lớp. Ví dụ, một SVM như vậy có thể so sánh lớp thứ k , được mã hóa là $+1$, với lớp thứ k , được mã hóa là -1 . Chúng ta phân loại một quan sát thử nghiệm bằng cách sử dụng từng bộ phân loại, và chúng ta đếm số lần quan sát thử nghiệm được gán cho mỗi trong số K lớp. Việc phân loại cuối cùng được thực hiện bằng cách gán quan sát thử nghiệm vào lớp mà nó được gán thường xuyên nhất trong số các lần so sánh này.

$\frac{K}{2}$ Phân loại theo cặp.

9.4.2 Phân loại Một đối với Tất cả

Phương pháp "một so với tất cả" (còn được gọi là "một so với phần còn lại") là một quy trình thay thế để áp dụng SVM trong trường hợp có $K > 2$ lớp. Chúng ta huấn luyện K SVM, mỗi lần so sánh một trong K lớp với $K - 1$ lớp còn lại. Gọi $\beta_0k, \beta_1k, \dots, \beta_{pk}$ là các tham số thu được từ việc huấn luyện một SVM so sánh lớp thứ k (được mã hóa là $+1$) với các lớp khác (được mã hóa là -1). Gọi x là một quan sát thử nghiệm. Chúng ta gán quan sát này vào lớp mà $\beta_0k + \beta_1kx + \beta_2kx^2 + \dots + \beta_{pk}x^p$ lớn nhất, vì điều này tương ứng với mức độ tin cậy cao rằng quan sát thử nghiệm thuộc về lớp thứ k chứ không phải bất kỳ lớp nào khác.

9.5 Mối quan hệ với hồi quy logistic

Khi máy hỗ trợ vectơ (SVM) được giới thiệu lần đầu vào giữa những năm 1990, chúng đã tạo ra một tiếng vang lớn trong cộng đồng thống kê và máy học. Điều này một phần là do hiệu suất tốt, tiếp thị hiệu quả, và cũng do phương pháp cơ bản của chúng có vẻ vừa mới lạ vừa bí ẩn. Ý tưởng tìm một siêu mặt phẳng phân tách dữ liệu tốt nhất có thể, đồng thời cho phép một số vi phạm đối với sự phân tách này, dường như khác biệt rõ rệt so với các phương pháp phân loại cổ điển, chẳng hạn như hồi quy logistic và phân tích phân biệt tuyến tính. Hơn nữa, ý tưởng sử dụng một hàm nhân (kernel) để mở rộng không gian đặc trưng nhằm phù hợp với các ranh giới lớp phi tuyến tính dường như là một đặc điểm độc đáo và có giá trị.

Tuy nhiên, kể từ thời điểm đó, mối liên hệ sâu sắc giữa SVM và các phương pháp thống kê cổ điển khác đã xuất hiện. Hóa ra, người ta có thể viết lại tiêu chí (9.12)–(9.15) để phù hợp với bộ phân loại vectơ hỗ trợ $f(X) = \beta_0 + \beta_1X_1 + \dots + \beta_pX_p$ như sau:

$$\max_{\beta_0, \beta_1, \dots, \beta_p} \left[\sum_{i=1}^N \min_{j=1, \dots, P} \{0, 1 - y_i f(x_i)\} + \lambda \sum_{j=1}^p \beta_j^2 \right], \quad (9.25)$$

trong đó λ là tham số điều chỉnh không âm. Khi λ lớn thì $\beta_1, \dots, \beta_p$ nhỏ, nhiều vi phạm giới hạn được chấp nhận hơn, và kết quả là một bộ phân loại có phương sai thấp nhưng độ lệch cao. Khi λ nhỏ thì sẽ có ít vi phạm giới hạn xảy ra; điều này dẫn đến một bộ phân loại có phương sai cao nhưng độ lệch thấp.



bộ phân loại. Do đó, giá trị nhỏ của λ trong (9.25) tương ứng với giá trị nhỏ của C trong (9.15). Lưu ý rằng $\sum_{j=1}^p \beta_j^2$ thuật ngữ trong (9.25) là thuật ngữ phạt sườn núi từ Mục 6.2.1, và đóng vai trò tương tự trong việc kiểm soát phương sai độ lệch. Sự đánh đổi đối với bộ phân loại vectơ hỗ trợ.

Giờ đây (9.25) mang hình thức “Thua + Phạt” mà chúng ta đã thấy nhiều lần xuyên suốt cuốn sách này:

giảm thiểu
$$\{L(X, y, \beta) + \lambda P(\beta)\} \text{ đối với } \beta_0, \beta_1, \dots, \beta_p \tag{9.26}$$

Trong (9.26), $L(X, y, \beta)$ là một hàm mất mát nào đó định lượng mức độ mà Mô hình, được tham số hóa bởi β , phù hợp với dữ liệu (X, y) , và $P(\beta)$ là một hình phạt. hàm trên vectơ tham số β có tác động được điều khiển bởi tham số điều chỉnh không âm λ . Ví dụ, cả hồi quy Ridge và Lasso đều... mang hình thức này với

$$L(X, y, \beta) = \sum_{i=1}^N \sum_{j=1}^p x_{ij} \beta_j y_i - \beta_0$$

và với $P(\beta) =$ cho hồi quy Ridge và $P(\beta) = \sum_{j=1}^p |\beta_j|$ cho j lasso. Trong trường hợp (9.25), hàm mất mát thay vào đó có dạng

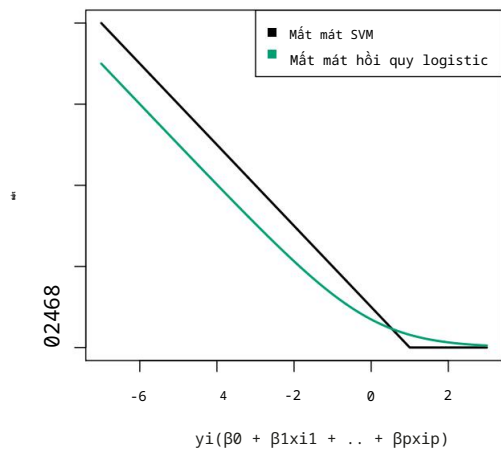
$$L(X, y, \beta) = \sum_{i=1}^N \max [0, 1 - y_i(\beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip})]$$

Đây được gọi là tổn thất bản lề, và được minh họa trong Hình 9.12. Tuy nhiên, thực tế cho thấy hàm tổn thất bản lề có liên quan mật thiết đến hàm tổn thất được sử dụng trong hồi quy logistic, cũng được thể hiện trong Hình 9.12.

Một đặc điểm thú vị của bộ phân loại vectơ hỗ trợ là chỉ có Các vectơ hỗ trợ đóng vai trò trong bộ phân loại thu được; các quan sát trên Phía bên phải của lề không ảnh hưởng đến nó. Điều này là do thực tế là... Hàm mất mát được thể hiện trong Hình 9.12 có giá trị chính xác bằng 0 đối với các quan sát mà tại đó $y_i(\beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip}) \geq 1$; những điều này tương ứng với các quan sát là ở phía bên phải của lề. Ngược lại, hàm mất mát cho logistic Đường hồi quy được thể hiện trong Hình 9.12 không hoàn toàn bằng không ở bất kỳ điểm nào. Nhưng nó rất nhỏ đối với các quan sát nằm xa ranh giới quyết định. Do đó, sự tương đồng giữa các hàm mất mát của chúng, hồi quy logistic và sự hỗ trợ Bộ phân loại vector thường cho kết quả rất giống nhau. Khi các lớp được phân loại tốt Khi tách biệt, SVM có xu hướng hoạt động tốt hơn so với hồi quy logistic; trong nhiều trường hợp khác nhau, SVM thường hoạt động tốt hơn. Trong các chế độ chồng chéo, hồi quy logistic thường được ưu tiên hơn.

Khi thuật toán phân loại vectơ hỗ trợ (SVM) và thuật toán SVM được giới thiệu lần đầu, người ta đã... nghĩ rằng tham số điều chỉnh C trong (9.15) là một tham số “gây phiền toái” không quan trọng có thể được đặt thành một giá trị mặc định nào đó, chẳng hạn như 1. Tuy nhiên, công thức “Mất mát + Hình phạt” (9.25) cho bộ phân loại vectơ hỗ trợ Điều này cho thấy rằng điều đó không đúng. Việc lựa chọn tham số điều chỉnh rất quan trọng. Điều này rất quan trọng và quyết định mức độ mô hình không khớp hoặc khớp quá mức với dữ liệu, như minh họa trong Hình 9.7 chẳng hạn.

3 Với cách biểu diễn tổn thất bản lề + phạt này, biên độ tương ứng với giá trị một, và độ rộng của lề được xác định bởi β_2 .



HÌNH 9.12. So sánh hàm mất mát của SVM và hồi quy logistic, theo hàm của $y_i(\beta_0 + \beta_1x_{i1} + \dots + \beta_{p_x}x_{ip_x})$. Khi $y_i(\beta_0 + \beta_1x_{i1} + \dots + \beta_{p_x}x_{ip_x})$ lớn hơn 1, thì hàm mất mát của SVM bằng 0, vì điều này tương ứng với một quan sát nằm ở phía bên phải của lề. Nhìn chung, hai hàm mất mát này có hành vi khá giống nhau.

Chúng ta đã xác định rằng bộ phân loại vectơ hỗ trợ (SVM) có mối liên hệ chặt chẽ với hồi quy logistic và các phương pháp thống kê hiện có khác. Liệu SVM có độc đáo trong việc sử dụng các hàm nhân (kernel) để mở rộng không gian đặc trưng nhằm đáp ứng các ranh giới lớp phi tuyến tính? Câu trả lời cho câu hỏi này là “không”. Chúng ta hoàn toàn có thể thực hiện hồi quy logistic hoặc nhiều phương pháp phân loại khác được trình bày trong cuốn sách này bằng cách sử dụng các hàm nhân phi tuyến tính; điều này có liên quan chặt chẽ đến một số phương pháp phi tuyến tính được trình bày trong Chương 7. Tuy nhiên, vì lý do lịch sử, việc sử dụng các hàm nhân phi tuyến tính phổ biến hơn nhiều trong bối cảnh của SVM so với bối cảnh của hồi quy logistic hoặc các phương pháp khác.

Mặc dù chúng ta chưa đề cập đến điều này ở đây, nhưng thực tế có một phần mở rộng của SVM dành cho hồi quy (tức là cho phân hồi định lượng chứ không phải định tính), được gọi là hồi quy vectơ hỗ trợ. Trong Chương 3, chúng ta đã thấy rằng hồi quy bình phương tối thiểu tìm kiếm các hệ số $\beta_0, \beta_1, \dots, \beta_p$ sao cho tổng bình phương phần dư nhỏ nhất có thể. (Nhớ lại từ Chương 3 rằng phần dư được định nghĩa là $y_i - \beta_0 - \beta_1x_{i1} - \dots - \beta_px_{ip}$.) Thay vào đó, hồi quy vectơ hỗ trợ tìm kiếm các hệ số làm giảm thiểu một loại mất mát khác, trong đó chỉ những phần dư có giá trị tuyệt đối lớn hơn một hằng số dương nào đó mới đóng góp vào hàm mất mát. Đây là sự mở rộng của biên độ được sử dụng trong các bộ phân loại vectơ hỗ trợ sang thiết lập hồi quy.

ứng hộ
vectơ
hồi quy

9.6 Bài thực hành: Máy hỗ trợ vectơ (Support Vector Machines)

Trong bài thực hành này, chúng ta sẽ sử dụng thư viện `sklearn.svm` để minh họa bộ phân loại vectơ hỗ trợ và máy vectơ hỗ trợ.

Chúng tôi nhập khẩu một số thư viện thông dụng của mình.

```
Trong [1]: import numpy as np from
matplotlib.pyplot import subplots, cm import
sklearn.model_selection as skm from ISLP import
load_data, confusion_table
```

Chúng tôi cũng thu thập các vật tư nhập khẩu mới cần thiết cho phòng thí nghiệm này.

```
Trong [2]: from sklearn.svm import SVC from
ISLP.svm import plot as plot_svm from sklearn.metrics
import RocCurveDisplay
```

Chúng ta sẽ sử dụng hàm `RocCurveDisplay.from_estimator()` để tạo ra
Một số biểu đồ ROC, sử dụng lệnh viết tắt `roc_curve`.

RocCurve
Hiển
thị.từ_estimator()

```
Trong [3]: roc_curve = RocCurveDisplay.from_estimator # viết tắt
```

9.6.1 Bộ phân loại vectơ hỗ trợ

Chúng ta sử dụng hàm `SupportVectorClassifier()` (viết tắt là `SVC()`) từ thư viện `sklearn` để huấn luyện bộ phân loại vectơ hỗ trợ cho một giá trị tham số `C` nhất định. Tham số `C` cho phép chúng ta chỉ định chi phí của việc vi phạm giới hạn. Khi tham số `C` nhỏ, thì giới hạn sẽ rộng và nhiều vectơ hỗ trợ sẽ nằm trên giới hạn hoặc vi phạm giới hạn. Khi tham số `C` lớn, thì giới hạn sẽ hẹp và sẽ có ít vectơ hỗ trợ nằm trên giới hạn hoặc vi phạm giới hạn.

SupportVector
Bộ phân loại()

Ở đây, chúng ta sẽ minh họa việc sử dụng `SVC()` trên một ví dụ hai chiều, để có thể vẽ ra ranh giới quyết định thu được. Chúng ta bắt đầu bằng cách tạo ra các quan sát, thuộc về hai lớp, và kiểm tra xem các lớp đó có thể phân tách tuyến tính hay không.

```
Trong [4]: rng = np.random.default_rng(1)
X = rng.standard_normal((50, 2)) y =
np.array([-1]*25+[1]*25)
X[y==1] += 1 fig,
ax = subplots(figsize=(8,8)) ax.scatter(X[:,0],
X[:,1],

c=y,
cmap=cm.coolwarm);
```

Chúng không phải vậy. Giờ chúng ta sẽ điều chỉnh bộ phân loại.

```
Trong [5]: svm_linear = SVC(C=10, kernel='linear') svm_linear.fit(X,
y)
```

```
Out[5]: SVC(C=10, kernel='linear')
```

Bộ phân loại vectơ hỗ trợ với hai đặc trưng có thể được trực quan hóa bằng cách vẽ đồ thị các giá trị của hàm quyết định của nó. Chúng tôi đã bao gồm một hàm cho việc này trong gói `ISLP` (lấy cảm hứng từ một ví dụ tương tự trong tài liệu `sklearn`).

hàm quyết
định

```
Trong [6]: fig, ax = subplots(figsize=(8,8))
          plot_svm(X,
                  y,
                  svm_linear,
                  ax=ax)
```

Ranh giới quyết định giữa hai lớp là tuyến tính (vì chúng ta đã sử dụng đối số `kernel='linear'`). Các vectơ hỗ trợ được đánh dấu bằng + và các quan sát còn lại được biểu diễn dưới dạng hình tròn.

Nếu thay vào đó chúng ta sử dụng giá trị nhỏ hơn cho tham số chi phí thì sao?

```
Trong [7]: svm_linear_small = SVC(C=0.1, kernel='linear')
          svm_linear_small.fit(X, y)
          fig,
          ax = subplots(figsize=(8,8))
          plot_svm(X,
                  y,
                  svm_linear_small, ax=ax)
```

Với giá trị tham số chi phí nhỏ hơn, ta thu được số lượng vectơ hỗ trợ lớn hơn, bởi vì biên độ lúc này rộng hơn. Đối với các hạt nhân tuyến tính, ta có thể trích xuất các hệ số của ranh giới quyết định tuyến tính như sau:

```
Trong [8]: svm_linear.coef_
```

```
Out[8]: array([[1.173 ,      0.7734]])
```

Vì máy vectơ hỗ trợ là một công cụ ước lượng trong `sklearn`, chúng ta có thể sử dụng các thiết bị thông thường để điều chỉnh nó.

```
Trong [9]: kfold = skm.KFold(5,
                           random_state=0,
                           shuffle=True)
          lưới = skm.GridSearchCV(svm_Tuyến tính,
                                  {'C':[0.001,0.01,0.1,1,5,10,100]}, refit=True,
                                  cv=kfold,
                                  scoring='accuracy')
          grid.fit(X, y)
          grid.best_params_
```

```
Ra[9]: {'C': 1}
```

Chúng ta có thể dễ dàng truy cập vào lỗi kiểm định chéo cho từng mô hình này trong `grid.cv_results_`. Kết quả in ra khá nhiều chi tiết, vì vậy chúng ta chỉ trích xuất kết quả về độ chính xác.

```
Trong [10]: grid.cv_results_[('mean_test_score')]
```

```
Out[10]: array([0.46, 0.46, 0.72, 0.74, 0.74, 0.74, 0.74])
```

Ta thấy rằng `C=1` mang lại độ chính xác kiểm định chéo cao nhất là 0,74, mặc dù độ chính xác là như nhau đối với một số giá trị của `C`. Bộ phân loại `grid.best_estimator_` có thể được sử dụng để dự đoán nhãn lớp trên một tập hợp các quan sát thử nghiệm. Hãy tạo một tập dữ liệu thử nghiệm.

```
Trong [11]: X_test = rng.standard_normal((20, 2))
           y_test = np.array([-1]*10+[1]*10)
           X_test[y_test==1] += 1
```

Bây giờ chúng ta dự đoán nhãn lớp của các quan sát thử nghiệm này. Ở đây chúng ta sử dụng Mô hình tốt nhất được chọn bằng phương pháp kiểm định chéo để đưa ra dự đoán.

```
Trong [12]: best_ = grid.best_estimator_ y_test_hat
           = best_.predict(X_test) confusion_table(y_test_hat,
           y_test)
```

```
Ra ngoài[12]:      Sự thật -1      1
              Dự đoán
                -1  8  4
                  126
```

Như vậy, với giá trị C này, 70% các quan sát thử nghiệm được phân loại chính xác. Vậy nếu chúng ta sử dụng $C=0,001$ thì sao?

```
Trong [13]: svm_ = SVC(C=0.001,
                    kernel='linear').fit(X, y) y_test_hat
           = svm_.predict(X_test) confusion_table(y_test_hat,
           y_test)
```

```
Ra ngoài[13]:      Sự thật -1      1
              Dự đoán
                -1  2  0
                  1  8 10
```

Trong trường hợp này, 60% các quan sát thử nghiệm được phân loại chính xác.

Bây giờ chúng ta xem xét trường hợp hai lớp có thể phân tách tuyến tính. Khi đó, chúng ta có thể tìm một siêu mặt phẳng phân tách tối ưu bằng cách sử dụng bộ ước lượng `SVC()`. Trước tiên, chúng ta tiếp tục phân tách hai lớp trong dữ liệu mô phỏng của mình sao cho chúng có thể phân tách tuyến tính:

```
Trong [14]: X[y==1] += 1.9; fig,
           ax = subplots(figsize=(8,8)) ax.scatter(X[:,0],
           X[:,1], c=y, cmap=cm.coolwarm);
```

Hiện tại, các quan sát chỉ còn cách nhau một khoảng tuyến tính rất nhỏ.

```
Trong [15]: svm_ = SVC(C=1e5, kernel='linear').fit(X, y) y_hat =
           svm_.predict(X)
           confusion_table(y_hat, y)
```

```
Ra ngoài[15]:      Sự thật -1  1
              Dự đoán
                -1 25  0
                  1   0 25
```

Chúng tôi huấn luyện bộ phân loại vectơ hỗ trợ và vẽ siêu mặt phẳng kết quả, sử dụng giá trị C rất lớn để đảm bảo không có quan sát nào bị phân loại sai.

```
Trong [16]: fig, ax = subplots(figsize=(8,8)) plot_svm(X,
           y,
           svm_,
           ax=ax)
```


390 9. Máy hỗ trợ vectơ

Thực tế là không có lỗi huấn luyện nào xảy ra và chỉ có ba vectơ hỗ trợ được sử dụng. Trên thực tế, giá trị lớn của C cũng có nghĩa là ba điểm hỗ trợ này nằm ở rìa và xác định nó. Người ta có thể tự hỏi bộ phân loại sẽ hoạt động tốt như thế nào trên dữ liệu thử nghiệm chỉ phụ thuộc vào ba điểm dữ liệu! Bây giờ chúng ta thử một giá trị nhỏ hơn của C .

```
Trong [17]: svm_ = SVC(C=0.1, kernel='linear').fit(X, y) y_hat = svm_.predict(X)
            confusion_table(y_hat, y)
```

```
Ra ngoài[17]:      Sự thật -1 1
              Dự đoán
              -1 25 0 0 25
                1
```

Khi sử dụng $C=0.1$, chúng ta lại không phân loại sai bất kỳ quan sát huấn luyện nào, đồng thời thu được biên độ rộng hơn nhiều và sử dụng mười hai vectơ hỗ trợ.

Những yếu tố này cùng nhau xác định hướng của ranh giới quyết định, và vì có nhiều yếu tố hơn nên nó ổn định hơn. Có vẻ như mô hình này sẽ hoạt động tốt hơn trên dữ liệu thử nghiệm so với mô hình với $C=1e5$ (và thực tế, một thí nghiệm đơn giản với một tập dữ liệu thử nghiệm lớn sẽ chứng minh điều này).

```
Trong [18]: fig, ax = subplots(figsize=(8,8)) plot_svm(X,
              y,
              svm_,
              ax=ax)
```

9.6.2 Máy hỗ trợ vectơ

Để phù hợp với SVM sử dụng hạt nhân phi tuyến tính, chúng ta lại sử dụng bộ ước lượng `SVC()`. Tuy nhiên, bây giờ chúng ta sử dụng một giá trị khác của tham số `kernel`. Để phù hợp với SVM với hạt nhân đa thức, chúng ta sử dụng `kernel="poly"`, và để phù hợp với SVM với hạt nhân xuyên tâm, chúng ta sử dụng `kernel="rbf"`. Trong trường hợp trước, chúng ta cũng sử dụng đối số `degree` để chỉ định bậc cho hạt nhân đa thức (đây là d trong (9.22)), và trong trường hợp sau, chúng ta sử dụng `gamma` để chỉ định giá trị của γ cho hạt nhân cơ sở xuyên tâm (9.24).

Trước tiên, chúng ta tạo ra một số dữ liệu với ranh giới lớp phi tuyến tính như sau:

```
Trong [19]: X = rng.standard_normal((200, 2))
            X[:100] += 2
            X[100:150] -= 2 y =
            np.array([1]*150+[2]*50)
```

Việc vẽ đồ thị dữ liệu cho thấy rõ ràng rằng ranh giới giữa các lớp thực sự không tuyến tính.

```
Trong [20]: fig, ax = subplots(figsize=(8,8)) ax.scatter(X[:,0],
              X[:,1],
              c=y,
              cmap=cm.coolwarm)
```

```
Out[20]: <matplotlib.collections.PathCollection at 0x7faa9ba52eb0 >
```

Dữ liệu được chia ngẫu nhiên thành nhóm huấn luyện và nhóm kiểm tra. Sau đó, chúng tôi sử dụng bộ ước lượng `SVC()` với hạt nhân xuyên tâm và $\gamma = 1$ để huấn luyện dữ liệu.

```
Trong [21]: (X_train,
           X_test,
           y_train,
           y_test) = skm.train_test_split(X,
                                           y,
                                           test_size=0.5,
                                           random_state=0)

svm_rbf = SVC(kernel="rbf", gamma=1, C=1)
svm_rbf.fit(X_train, y_train)
```

Đồ thị cho thấy SVM thu được có ranh giới phi tuyến tính rõ rệt.

```
Trong [22]: fig, ax = subplots(figsize=(8,8))
           plot_svm(X_train,
                   y_train,
                   svm_rbf,
                   ax=ax)
```

Từ hình vẽ, ta có thể thấy có khá nhiều lỗi huấn luyện trong quá trình huấn luyện SVM này. Nếu tăng giá trị của C , ta có thể giảm số lỗi huấn luyện. Tuy nhiên, điều này lại dẫn đến đường biên quyết định không đều hơn, có nguy cơ dẫn đến hiện tượng quá khớp dữ liệu.

```
Trong [23]: svm_rbf = SVC(kernel="rbf", gamma=1, C=1e5)
           svm_rbf.fit(X_train, y_train)
           fig, ax = subplots(figsize=(8,8))
           plot_svm(X_train,
                   y_train, svm_rbf,
                   ax=ax)
```

Chúng ta có thể thực hiện kiểm định chéo bằng cách sử dụng `skm.GridSearchCV()` để chọn giá trị γ và C tốt nhất cho SVM với hạt nhân xuyên tâm:

```
Trong [24]: kfold = skm.KFold(5,
                             random_state=0,
                             shuffle=True)

grid = skm.GridSearchCV(svm_rbf, {'C':
                                [0.1, 1, 10, 100, 1000], 'gamma':
                                [0.5, 1, 2, 3, 4]},
                        refit=True,
                        cv=kfold,
                        scoring='accuracy');

grid.fit(X_train, y_train)
grid.best_params_
```

```
Out[24]: {'C': 100, 'gamma': 1}
```

Sự lựa chọn tham số tối ưu nhất trong phương pháp kiểm định chéo năm lần đạt được ở $C=1$ và $\gamma=0,5$, mặc dù một số giá trị khác cũng cho kết quả tương tự.

```
Trong [25]: best_svm = grid.best_estimator_
           fig, ax = subplots(figsize=(8,8))
           plot_svm(X_train,
```

```
y_train,
best_svm,
ax=ax)

y_hat_test = best_svm.predict(X_test)
confusion_table(y_hat_test, y_test)
```

Ra ngoài[25]: Sự thật 1 2
Dự đoán
 1 69 6
 2 6 19

Với các tham số này, 12% số quan sát trong thử nghiệm bị phân loại sai bởi thuật toán SVM này.

9.6.3 Đường cong ROC

Các thuật toán SVM và bộ phân loại vectơ hỗ trợ đưa ra nhãn lớp cho mỗi quan sát. Tuy nhiên, cũng có thể thu được các giá trị phù hợp cho mỗi quan sát, đó là các điểm số được sử dụng để thu được nhãn lớp. Ví dụ, trong trường hợp bộ phân loại vectơ hỗ trợ, giá trị phù hợp cho một quan sát $X = (X_1, X_2, \dots, X_p)^T$ có dạng $\beta^0 + \beta^1 X_1 + \beta^2 X_2 + \dots + \beta^p X_p$. Đối với SVM với hạt nhân phi tuyến tính, phương trình tạo ra giá trị phù hợp được đưa ra trong (9.23). Dấu của giá trị phù hợp xác định quan sát nằm ở phía nào của ranh giới quyết định. Do đó, mối quan hệ giữa giá trị phù hợp và dự đoán lớp cho một quan sát nhất định rất đơn giản: nếu giá trị phù hợp lớn hơn 0 thì quan sát được gán cho một lớp, và nếu nó nhỏ hơn 0 thì nó được gán cho lớp khác.

Bằng cách thay đổi ngưỡng này từ 0 thành một giá trị dương nào đó, chúng ta làm lệch phân loại theo hướng có lợi cho một lớp so với lớp kia. Bằng cách xem xét một loạt các ngưỡng này, cả dương và âm, chúng ta tạo ra các thành phần cho biểu đồ ROC. Chúng ta có thể truy cập các giá trị này bằng cách gọi phương thức `decision_function()` của bộ ước lượng SVM đã được huấn luyện.

Hàm `ROCCurveDisplay.from_estimator()` (mà chúng tôi đã viết tắt thành `roc_curve()`) sẽ tạo ra một đồ thị đường cong ROC. Nó nhận một ước lượng đã được hiệu chỉnh làm đối số đầu tiên, tiếp theo là ma trận mô hình X và nhãn y . Tên đối số được sử dụng trong chú thích, trong khi màu sắc được sử dụng cho màu của đường kẻ. Kết quả được vẽ trên đối tượng trục `ax` của chúng ta .

```
.function_decision()
roc_curve()
```

Trong [26]:

```
fig, ax = subplots(figsize=(8,8))
roc_curve(best_svm,
           X_train,
           y_train,
           name='Training',
           color='r',
           ax=ax);
```

Trong ví dụ này, SVM dường như cung cấp các dự đoán chính xác. Bằng cách tăng γ , chúng ta có thể tạo ra một mô hình phù hợp linh hoạt hơn và tạo ra những cải tiến hơn nữa về độ chính xác.

Trong [27]:

```
svm_flex = SVC(kernel="rbf",
                gamma=50,
```

```

C=1)
svm_flex.fit(X_train, y_train) fig, ax
= subplots(figsize=(8,8)) roc_curve(svm_flex,
X_train, y_train,

name='Training $\gamma=50$',
color='r',
ax=ax);

```

Tuy nhiên, tất cả các đường cong ROC này đều được vẽ trên dữ liệu huấn luyện. Chúng ta thực sự quan tâm hơn đến mức độ chính xác dự đoán trên dữ liệu kiểm tra. Khi tính toán các đường cong ROC trên dữ liệu kiểm tra, mô hình với $\gamma = 0,5$ dường như cho kết quả chính xác nhất.

Trong [28]:

```

roc_curve(svm_flex,
X_test,
y_test,
name='Test $\gamma=50$',
color='b',
ax=ax)

```

quả sung;

Hãy cùng xem xét mô hình SVM đã được tinh chỉnh của chúng ta.

Trong [29]:

```

fig, ax = subplots(figsize=(8,8)) for (X_,
y_, c, name) in zip(
(X_train, X_test),
(y_train, y_test),
('r', 'b'),
('CV được tinh chỉnh để phục vụ đào tạo',
'CV đã được tinh chỉnh trong quá trình thử nghiệm')):
roc_curve(best_svm,
X_,
y_,
tên=tên,
ax=ax,
màu=c)

```

9.6.4 SVM với nhiều lớp

Nếu phản hồi là một yếu tố chứa nhiều hơn hai cấp độ, thì hàm `SVC()` sẽ thực hiện phân loại đa lớp bằng cách sử dụng phương pháp một đối một (khi `decision_function_shape=='ovo'`) hoặc một đối với phần còn lại (khi `decision_function_shape=='ovr'`). Chúng ta sẽ khám phá ngắn gọn thiết lập này bằng cách tạo ra một lớp quan sát thứ ba.

Trong [30]:

```

rng = np.random.default_rng(123)
X = np.vstack([X, rng.standard_normal((50, 2))]) y =
np.hstack([y, [0]*50])
X[y==0,1] += 2
fig, ax = subplots(figsize=(8,8))
ax.scatter(X[:,0], X[:,1], c=y, cmap=cm.coolwarm);

```

4. Chiến thuật "một chơi tất cả" cũng được biết đến với tên gọi "một chơi tất cả".

Bây giờ chúng ta sẽ sử dụng mô hình SVM để huấn luyện dữ liệu:

```
Trong [31]: svm_rbf_3 = SVC(kernel="rbf",
                        C=10,
                        gamma=1,
                        decision_function_shape='ovo');

svm_rbf_3.fit(X, y)
hình, trục = subplots(kích thước hình=(8,8))
plot_svm(X,
        y,
        svm_rbf_3,
        scatter_cmap=cm.tab10,
        ax=ax)
```

Thư viện `sklearn.svm` cũng có thể được sử dụng để thực hiện hồi quy vectơ hỗ trợ với phân hồi dạng số bằng cách sử dụng bộ ước lượng `SupportVector-Regression()`.

SupportVector
Hồi quy()

9.6.5 Ứng dụng vào dữ liệu biểu hiện gen

Bây giờ chúng ta sẽ xem xét tập dữ liệu `Khan`, bao gồm một số mô. Các mẫu tương ứng với bốn loại khối u tế bào tròn nhỏ màu xanh lam khác nhau. Đối với mỗi mẫu mô, dữ liệu về biểu hiện gen đều có sẵn. Tập dữ liệu bao gồm dữ liệu huấn luyện (`xtrain` và `ytrain`) và dữ liệu kiểm thử. `xtest` và `ytest`. Chúng tôi xem xét khía cạnh của dữ liệu:

```
Trong [32]: Khan = load_data('Khan')
Khan['xtrain'].shape, Khan['xtest'].shape
```

Ra[32]: ((63, 2308), (20, 2308))

Bộ dữ liệu này bao gồm các phép đo biểu hiện của 2.308 gen. Tập dữ liệu huấn luyện và tập dữ liệu kiểm tra lần lượt bao gồm 63 và 20 quan sát. Chúng tôi sẽ sử dụng phương pháp hỗ trợ vectơ để dự đoán loại ung thư bằng cách sử dụng các phép đo biểu hiện gen. Trong tập dữ liệu này, có một số lượng rất lớn. các đặc điểm tương đối so với số lượng quan sát. Điều này cho thấy rằng chúng ta nên sử dụng nhân tuyến tính, vì tính linh hoạt bổ sung mà nó mang lại sẽ tạo ra. Việc sử dụng nhân đa thức hoặc nhân xuyên tâm là không cần thiết.

```
Trong [33]: khan_linear = SVC(kernel='linear', C=10)
khan_linear.fit(Khan['xtrain'], Khan['ytrain'])
confusion_table(khan_linear.predict(Khan['xtrain']),
                Khan['ytrain'])
```

Ra ngoài[33]:

Sự thật	1	2	3	4
Dự đoán				
1	8000			
2	0	23	0	0
3	0	0	12	0
4	0	0	0	20

Chúng ta thấy rằng không có lỗi nào trong quá trình huấn luyện. Thực tế, điều này không có gì đáng ngạc nhiên. vì số lượng biến số lớn so với số lượng quan sát. Điều này ngụ ý rằng việc tìm ra các siêu mặt phẳng phân tách hoàn toàn các lớp là điều dễ dàng.

Chúng tôi quan tâm hơn đến hiệu năng của bộ phân loại vectơ hỗ trợ trên các quan sát thử nghiệm.

```
Trong [34]: confusion_table(khan_linear.predict(Khan['xtest']),
                           Khan['ytest'])
```

```
Ra ngoài[34]:      Sự thật      1 2 3 4
Dự đoán
1 3000 2 0620 3 0040 4
0005
```

Chúng ta thấy rằng việc sử dụng $C=10$ dẫn đến hai lỗi trong tập dữ liệu kiểm thử trên các dữ liệu này.

9.7 Bài tập

Khái niệm

1. Bài toán này liên quan đến siêu mặt phẳng trong không gian hai chiều.

(a) Vẽ siêu mặt phẳng $1+3X_1 - X_2 = 0$. Chỉ ra tập hợp các điểm mà $1+3X_1 - X_2 > 0$, cũng như tập hợp các điểm mà $1+3X_1 - X_2 < 0$.

(b) Trên cùng một đồ thị, hãy phác họa siêu mặt phẳng $2 + X_1 + 2X_2 = 0$. Hãy chỉ ra tập hợp các điểm mà tại đó $2 + X_1 + 2X_2 > 0$, cũng như tập hợp các điểm mà tại đó $2 + X_1 + 2X_2 < 0$.

2. Chúng ta đã thấy rằng trong không gian $p = 2$ chiều, ranh giới quyết định tuyến tính có dạng $\beta_0 + \beta_1 X_1 + \beta_2 X_2 = 0$. Bây giờ chúng ta sẽ nghiên cứu ranh giới quyết định phi tuyến tính.

(a) Vẽ đường cong

$$(1 + X_1)^2 + (2 - X_2)^2 = 4.$$

(b) Trên bản vẽ của bạn, hãy chỉ ra tập hợp các điểm mà tại đó

$$(1 + X_1)^2 + (2 - X_2)^2 > 4,$$

cũng như tập hợp các điểm mà

$$(1 + X_1)^2 + (2 - X_2)^2 \leq 4.$$

(c) Giả sử bộ phân loại gán một quan sát vào lớp màu xanh lam.

như sau

$$(1 + X_1)^2 + (2 - X_2)^2 > 4,$$

và thuộc lớp màu đỏ trong trường hợp khác. Quan sát $(0, 0)$ được phân loại vào lớp nào? $(-1, 1)$? $(2, 2)$? $(3, 8)$? (d) Hãy lập

luận rằng trong khi ranh giới quyết định trong (c) không tuyến tính theo X_1 và X_2 , thì nó lại tuyến tính theo X_1, X_2, X_2^2 và X_2 .

2 .

3. Ở đây, chúng ta sẽ khám phá bộ phân loại biên độ tối đa trên một tập dữ liệu giả lập.

(a) Chúng ta được cho $n = 7$ quan sát trong $p = 2$ chiều. Đối với mỗi Quan sát cho thấy có một nhãn lớp liên quan.

Quan sát.	X1	X2	Y
1	3	4	Đỏ
2	2	2	Đỏ
3	4	4	Đỏ
4	1	4	Đỏ
5	2	1	Màu xanh lam
6	4	3	màu xanh lam
7	4	1	Màu xanh lam

Phác thảo các quan sát.

- (b) Phác thảo siêu mặt phẳng phân tách tối ưu và cung cấp phương trình. tion cho siêu mặt phẳng này (có dạng (9.1)).
- (c) Mô tả quy tắc phân loại cho bộ phân loại biên độ tối đa. Nó nên có nội dung tương tự như "Phân loại thành màu đỏ nếu..." Nếu $\beta_0 + \beta_1 X_1 + \beta_2 X_2 > 0$, thì phân loại là Xanh lam nếu không phải vậy." Cung cấp các giá trị cho β_0 , β_1 và β_2 .
- (d) Trên bản vẽ của bạn, hãy chỉ ra lề tối đa. Siêu mặt phẳng.
- (e) Chỉ ra các vectơ hỗ trợ cho bộ phân loại biên độ tối đa.
- (f) Hãy lập luận rằng một sự thay đổi nhỏ của quan sát thứ bảy sẽ không ảnh hưởng đến siêu mặt phẳng biên độ tối đa.
- (g) Vẽ một siêu mặt phẳng không phải là siêu mặt phẳng phân tách tối ưu và cung cấp phương trình của siêu mặt phẳng này.
- (h) Vẽ thêm một quan sát nữa trên đồ thị sao cho hai quan sát đó trùng khớp với nhau. Các lớp không còn phân tách được bởi một siêu mặt phẳng nữa.

Đã áp dụng

4. Tạo một tập dữ liệu mô phỏng hai lớp với 100 quan sát và hai đặc điểm trong đó có sự phân tách rõ ràng nhưng không tuyến tính giữa hai lớp. Chứng minh rằng trong thiết lập này, một vectơ hỗ trợ máy có nhân đa thức (với bậc lớn hơn 1) hoặc một Thuật toán Radial Kernel sẽ cho kết quả tốt hơn thuật toán Support Vector Classifier trên dữ liệu huấn luyện. Vậy kỹ thuật nào cho kết quả tốt nhất trên dữ liệu kiểm thử? Vẽ biểu đồ và báo cáo tỷ lệ lỗi trong quá trình huấn luyện và kiểm tra để hỗ trợ. Những khẳng định của bạn.
5. Chúng ta đã thấy rằng chúng ta có thể điều chỉnh SVM với một hạt nhân phi tuyến tính để thực hiện phân loại bằng cách sử dụng ranh giới quyết định phi tuyến tính. Chúng ta sẽ Bây giờ, chúng ta thấy rằng chúng ta cũng có thể thu được ranh giới quyết định phi tuyến tính bằng cách thực hiện hồi quy logistic bằng cách sử dụng các phép biến đổi phi tuyến tính của đặc trưng.

- (a) Tạo một tập dữ liệu với $n = 500$ và $p = 2$, sao cho các quan sát thuộc về hai lớp với ranh giới quyết định bậc hai giữa chúng. Ví dụ, bạn có thể thực hiện như sau:

```
rng = np.random.default_rng(5) x1
= rng.uniform(size=500) - 0.5 x2 =
rng.uniform(size=500) - 0.5 y = x1**2
- x2**2 > 0
```

- (b) Vẽ biểu đồ các quan sát, tô màu theo nhãn lớp của chúng.
Đồ thị của bạn nên hiển thị X_1 trên trục x và X_2 trên trục y .

- (c) Áp dụng mô hình hồi quy logistic cho dữ liệu, sử dụng X_1 và X_2 làm các yếu tố dự báo.

- (d) Áp dụng mô hình này vào dữ liệu huấn luyện để thu được nhãn lớp dự đoán trước cho mỗi quan sát huấn luyện. Vẽ đồ thị các quan sát, tô màu theo nhãn lớp dự đoán. Đường ranh giới quyết định phải là đường thẳng.

- (e) Bây giờ hãy áp dụng mô hình hồi quy logistic cho dữ liệu bằng cách sử dụng các hàm phi tuyến của X_1 và X_2 làm biến dự đoán (ví dụ: X_2^2 , $X_1 \times X_2$, $\log(X_2)$, v.v.). (f) Áp dụng mô hình

này cho dữ liệu huấn luyện để thu được nhãn lớp dự đoán cho mỗi quan sát huấn luyện. Vẽ đồ thị các quan sát, tô màu theo nhãn lớp dự đoán. Đường ranh giới quyết định phải rõ ràng là phi tuyến tính. Nếu không, hãy lặp lại (a)-(e) cho đến khi bạn tìm được một ví dụ trong đó các nhãn lớp dự đoán rõ ràng là phi tuyến tính. (g) Áp dụng bộ phân loại vectơ hỗ trợ cho dữ liệu với X_1 và X_2 làm biến dự đoán. Thu được dự đoán lớp cho mỗi quan sát huấn luyện. Vẽ đồ thị các quan sát, tô màu theo nhãn lớp dự đoán.

- (h) Huấn luyện mô hình SVM sử dụng hạt nhân phi tuyến tính cho dữ liệu. Thu được dự đoán lớp cho mỗi quan sát huấn luyện. Vẽ biểu đồ các quan sát, tô màu theo nhãn lớp dự đoán. (i) Nhận xét về kết quả của bạn.

6. Ở cuối Mục 9.6.1, người ta khẳng định rằng trong trường hợp dữ liệu chỉ vừa đủ phân tách tuyến tính, một bộ phân loại vectơ hỗ trợ với giá trị C nhỏ, nếu phân loại sai một vài quan sát huấn luyện, có thể hoạt động tốt hơn trên dữ liệu kiểm tra so với một bộ phân loại với giá trị C lớn mà không phân loại sai bất kỳ quan sát huấn luyện nào. Bây giờ bạn sẽ nghiên cứu khẳng định này.

- (a) Tạo dữ liệu hai lớp với $p = 2$ sao cho các lớp
Chúng chỉ tách biệt nhau một cách tuyến tính rất nhỏ.

- (b) Tính toán tỷ lệ lỗi kiểm định chéo cho các bộ phân loại vectơ hỗ trợ với một loạt các giá trị C . Có bao nhiêu quan sát huấn luyện bị phân loại sai cho mỗi giá trị C được xem xét, và điều này liên quan như thế nào đến các lỗi kiểm định chéo thu được?

9. Máy hỗ trợ vectơ

(c) Tạo một tập dữ liệu kiểm thử phù hợp và tính toán các lỗi kiểm thử tương ứng với mỗi giá trị của C đã xem xét. Giá trị nào của C dẫn đến số lỗi kiểm thử ít nhất, và điều này so sánh như thế nào với các giá trị của C tạo ra số lỗi huấn luyện ít nhất và số lỗi kiểm định chéo ít nhất?

(d) Thảo luận về kết quả của bạn.

7. Trong bài toán này, bạn sẽ sử dụng các phương pháp hỗ trợ vector để dự đoán xem một chiếc xe cụ thể có mức tiêu hao nhiên liệu cao hay thấp dựa trên tập dữ liệu *Auto*.

(a) Tạo một biến nhị phân nhận giá trị 1 đối với những xe có mức tiêu hao nhiên liệu trên mức trung vị và giá trị 0 đối với những xe có mức tiêu hao nhiên liệu dưới mức trung vị.

(b) Áp dụng bộ phân loại vectơ hỗ trợ cho dữ liệu với các giá trị C khác nhau để dự đoán xem một chiếc xe có mức tiêu hao nhiên liệu cao hay thấp.

Hãy báo cáo các lỗi kiểm định chéo liên quan đến các giá trị khác nhau của tham số này. Nhận xét về kết quả của bạn. Lưu ý rằng bạn cần huấn luyện bộ phân loại mà không có biến mức tiêu hao nhiên liệu để thu được kết quả hợp lý.

(c) Bây giờ hãy lặp lại (b), lần này sử dụng SVM với hạt nhân cơ sở xuyên tâm và đa thức, với các giá trị γ , β và C khác nhau. Nhận xét về kết quả của bạn. (d) Vẽ một số đồ thị để chứng minh cho

nhận định của bạn trong (b) và (c).

Gợi ý: Trong phòng thí nghiệm, chúng tôi đã sử dụng hàm `plot_svm()` cho các SVM đã được hiệu chỉnh.

Khi $p > 2$, bạn có thể sử dụng các tính năng đối số từ khóa để tạo biểu đồ hiển thị từng cặp biến một.

8. Bài toán này liên quan đến tập dữ liệu *OJ*, một phần của *ISLP*.

bầu kiện.

(a) Tạo một tập huấn luyện chứa một mẫu ngẫu nhiên gồm 800 quan sát và một tập kiểm tra chứa các quan sát còn lại.

(b) Hãy huấn luyện một bộ phân loại vectơ hỗ trợ trên dữ liệu huấn luyện với hệ số $C = 0,01$, trong đó biến *Purchase* là biến phản hồi và các biến khác là biến dự đoán. Có bao nhiêu điểm hỗ trợ?

(c) Tỷ lệ lỗi huấn luyện và kiểm tra là bao nhiêu? (d) Sử dụng

phương pháp kiểm định chéo để chọn giá trị C tối ưu. Hãy xem xét các giá trị trong phạm vi từ 0,01 đến 10.

(e) Tính toán tỷ lệ lỗi huấn luyện và kiểm tra bằng cách sử dụng giá trị mới này. cho C .

(f) Lặp lại các bước (b) đến (e) bằng cách sử dụng máy vectơ hỗ trợ với hạt nhân xuyên tâm. Sử dụng giá trị mặc định cho γ . (g) Lặp lại các bước (b) đến

(e) bằng cách sử dụng máy vectơ hỗ trợ với hạt nhân đa thức. Đặt $\beta = 2$.

(h) Nhìn chung, phương pháp nào cho kết quả tốt nhất về vấn đề này? dữ liệu?

10

Học sâu



Chương này đề cập đến chủ đề quan trọng về học sâu (deep learning). Tại thời điểm viết chương này (năm 2020), học sâu là một lĩnh vực nghiên cứu rất sôi động trong ngành học máy.

Cộng đồng học tập và trí tuệ nhân tạo. Nền tảng của sự hiểu biết sâu sắc.

Học tập chính là mạng lưới thần kinh.

Mạng nơ-ron trở nên nổi tiếng vào cuối những năm 1980. Có rất nhiều sự hào hứng và một chút

mạng
nơ-ron

cường điệu liên quan đến phương pháp này, và chúng

là động lực thúc đẩy sự ra đời của các Hệ thống Xử lý Thông tin Thần kinh phổ biến.

các cuộc họp (NeurIPS, trước đây là NIPS) được tổ chức hàng năm, thường ở những địa điểm kỳ lạ. những nơi như khu nghỉ dưỡng trượt tuyết. Tiếp theo đó là giai đoạn tổng hợp, trong đó...

Các đặc tính của mạng nơ-ron đã được các nhà học máy, nhà toán học và nhà thống kê phân tích; các thuật toán được cải tiến và phương pháp luận được ổn định. Sau đó, các thuật toán SVM, boosting và rừng ngẫu nhiên xuất hiện.

và mạng nơ-ron dần mất đi sự ưa chuộng. Một phần lý do là...

Mạng nơ-ron đòi hỏi rất nhiều sự tinh chỉnh, trong khi các phương pháp mới thì khác.

Tự động hơn. Ngoài ra, trên nhiều bài toán, các phương pháp mới cho kết quả tốt hơn.

Các mạng nơ-ron được huấn luyện kém. Đây là hiện trạng trong thập kỷ đầu tiên. trong thiên niên kỷ mới.

Tuy nhiên, trong suốt thời gian đó, một nhóm cốt lõi những người đam mê mạng nơ-ron đã...

đẩy mạnh công nghệ của họ hơn nữa trên các kiến trúc điện toán ngày càng lớn hơn và tập dữ liệu. Mạng nơ-ron xuất hiện trở lại sau năm 2010 với tên gọi mới là mạng nơ-ron sâu.

học tập, với kiến trúc mới, các tính năng bổ sung và một chuỗi

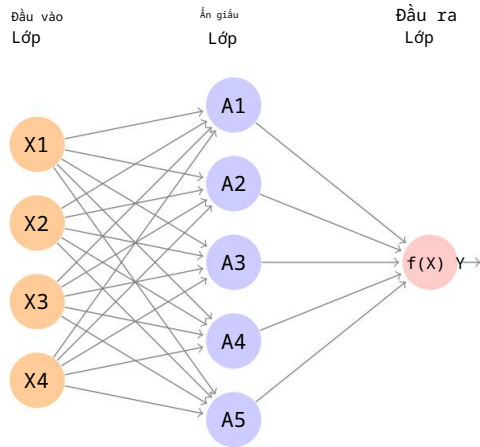
có nhiều câu chuyện thành công trong một số vấn đề chuyên biệt như phân loại hình ảnh và video, mô hình hóa giọng nói và văn bản. Nhiều người trong lĩnh vực này tin rằng vấn đề chính là...

Lý do cho những thành công này là do sự sẵn có của các tập dữ liệu huấn luyện ngày càng lớn.

Điều này trở nên khả thi nhờ việc ứng dụng rộng rãi công nghệ số hóa trong khoa học và công nghiệp.

Trong chương này, chúng ta sẽ thảo luận về những kiến thức cơ bản của mạng nơ-ron và học sâu, sau đó đi sâu vào một số lĩnh vực chuyên biệt để giải quyết các vấn đề cụ thể, chẳng hạn như...

như mạng nơ-ron tích chập (CNN) để phân loại hình ảnh và mạng nơ-ron hồi quy (RNN) cho chuỗi thời gian và các chuỗi khác. Chúng tôi



HÌNH 10.1. Mạng nơ-ron với một lớp ẩn duy nhất. Lớp ẩn tính toán các kích hoạt $A_k = h_k(X)$ là các phép biến đổi phi tuyến của tuyến tính các tổ hợp của các đầu vào $X_1, X_2, \dots, X_p$. Do đó, các A_k này không được quan sát trực tiếp. Các hàm $h_k(\cdot)$ không được cố định trước mà được học trong quá trình học. huấn luyện mạng. Lớp đầu ra là một mô hình tuyến tính sử dụng các kích hoạt A_k này làm đầu vào, dẫn đến một hàm $f(X)$.

Tôi cũng sẽ trình bày các mô hình này bằng cách sử dụng gói `torch` của `Python`, cùng với... với một số gói hỗ trợ.

Nội dung chương này có phần khó hơn so với các chương khác. trong cuốn sách này.

10.1 Mạng nơ-ron đơn lớp

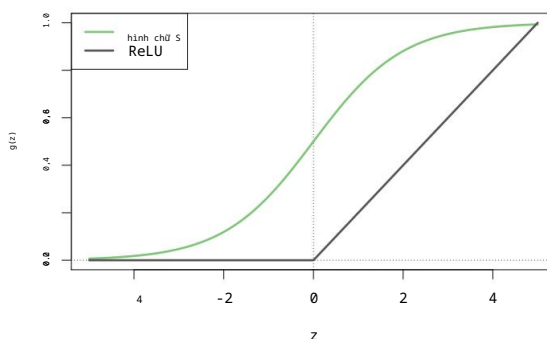
Một mạng nơ-ron nhận đầu vào là một vectơ gồm p biến $X = (X_1, X_2, \dots, X_p)$ và xây dựng một hàm phi tuyến $f(X)$ để dự đoán phản hồi Y . Chúng ta có đã xây dựng các mô hình dự đoán phi tuyến tính trong các chương trước, sử dụng cây quyết định và thuật toán boosting. và các mô hình cộng tính tổng quát. Điều gì phân biệt mạng nơ-ron với... Các phương pháp này là cấu trúc đặc thù của mô hình. Hình 10.1 cho thấy Một mạng nơ-ron truyền thẳng đơn giản để mô hình hóa phản hồi định lượng sử dụng $p = 4$ biến dự đoán. Theo thuật ngữ của mạng nơ-ron, bốn đặc trưng $X_1, \dots, X_4$ tạo thành các đơn vị trong lớp đầu vào. Các mũi tên chỉ ra Mỗi đầu vào từ lớp đầu vào sẽ được đưa vào từng lớp đầu vào ẩn K . đơn vị (chúng ta được chọn K ; ở đây chúng ta chọn 5). Mô hình mạng nơ-ron có dạng

phản hồi tiên
thần kinh
mạng
đơn vị ẩn

$$\begin{aligned} f(X) &= \beta_0 + \sum_{k=1}^K \beta_k h_k(X) \\ &= \beta_0 + \sum_{k=1}^K \beta_k g(\text{tuần } 0 + \sum_{j=1}^p w_{kj} X_j). \end{aligned} \tag{10.1}$$

Quá trình này được xây dựng qua hai bước. Đầu tiên, K giá trị kích hoạt A_k , $k = 1, \dots, K$, trong lớp ẩn được tính toán như các hàm của các đặc trưng đầu vào $X_1, \dots, X_p$,

$$A_k = h_k(X) = g(w_k^0 + \sum_{j=1}^p w_{kj} X_j), \tag{10.2}$$



HÌNH 10.2. Các hàm kích hoạt. Hàm ReLU tuyến tính từng phần được ưa chuộng vì hiệu quả và khả năng tính toán của nó. Chúng tôi đã thu nhỏ nó xuống một hệ số là Năm điểm để dễ so sánh.

trong đó $g(z)$ là một hàm kích hoạt phi tuyến được xác định trước.

Ta có thể coi mỗi A_k như một phép biến đổi khác nhau $h_k(X)$ của bản gốc. các tính năng, tương tự như các hàm cơ sở của Chương 7. Các kích hoạt K này Từ lớp ẩn, tín hiệu được đưa vào lớp đầu ra, dẫn đến kết quả là...

chức năng
kích hoạt

$$f(X) = \beta_0 + \sum_{k=1}^K \beta_k A_k, \quad (10.3)$$

một mô hình hồi quy tuyến tính trong $K = 5$ kích hoạt. Tất cả các tham số $\beta_0, \dots, \beta_K$ và $w_{10}, \dots, w_{Kp}$ cần được ước tính từ dữ liệu. Trong giai đoạn đầu Trong các trường hợp mạng nơ-ron, hàm kích hoạt sigmoid được ưa chuộng.

hình chữ S

$$g(z) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{-z}}, \quad (10.4)$$

Đây là hàm tương tự được sử dụng trong hồi quy logistic để chuyển đổi một mô hình tuyến tính. hàm số được chuyển đổi thành xác suất nằm giữa 0 và 1 (xem Hình 10.2). Hàm kích hoạt ReLU (rectified linear unit), được ưa chuộng trong các mạng nơ-ron hiện đại, có dạng như sau:

ReLU
đã sửa chữa
đơn vị tuyến tính

$$g(z) = \begin{cases} 0 & \text{nếu } z < 0 \\ z & \text{nếu không thì.} \end{cases} \quad (10.5)$$

Hàm kích hoạt ReLU có thể được tính toán và lưu trữ hiệu quả hơn so với...

Kích hoạt sigmoid. Mặc dù ngưỡng của nó ở mức 0, nhưng vì chúng ta áp dụng nó cho một hàm tuyến tính (10.2) hằng số w_{k0} sẽ dịch chuyển điểm uốn này.

Nói tóm lại, mô hình được minh họa trong Hình 10.1 đưa ra năm đặc điểm mới.

bằng cách tính toán năm tổ hợp tuyến tính khác nhau của X , rồi nên chúng lại.

mỗi cái được biến đổi thông qua một hàm kích hoạt $g(\cdot)$. Mô hình cuối cùng có tính tuyến tính đối với các biến dẫn xuất này.

Tên gọi mạng nơ-ron ban đầu xuất phát từ suy nghĩ về những cấu trúc ẩn này. các đơn vị tương tự như các tế bào thần kinh trong não – giá trị của các kích hoạt $A_k = h_k(X)$ gần bằng một là đang phát xung, trong khi những giá trị gần bằng không thì im lặng. (sử dụng hàm kích hoạt sigmoid).

Tính phi tuyến tính trong hàm kích hoạt $g(\cdot)$ là rất quan trọng, vì nếu không có nó thì sẽ không thể xảy ra hiện tượng này. mô hình $f(X)$ trong (10.1) sẽ sụp đổ thành một mô hình tuyến tính đơn giản trong

402 10. Học sâu

$x_1, \dots, x_p$. Hơn nữa, việc có một hàm kích hoạt phi tuyến tính cho phép mô hình nắm bắt được các phi tuyến tính phức tạp và các hiệu ứng tương tác. Hãy xem xét một ví dụ rất đơn giản với $p = 2$ biến đầu vào $X = (x_1, x_2)$, và $K = 2$ đơn vị ẩn $h_1(X)$ và $h_2(X)$ với $g(z) = z^2$. Chúng ta xác định các tham số khác như sau:

$$\begin{aligned} \beta_0 &= 0, \beta_1 = 4, \beta_2 = \frac{1}{4}, \\ w_{10} &= 0, w_{11} = 1, w_{12} = 1, w_{20} = 0, w_{21} = 1, w_{22} \\ &= 1. \end{aligned} \quad (10.6)$$

Từ (10.2), điều này có nghĩa là

$$\begin{aligned} h_1(X) &= (\beta_0 + \beta_1 x_1 + \beta_2 x_2)^2, \quad h_2(X) = \\ &= (\beta_0 + \beta_1 x_1 - \beta_2 x_2)^2. \end{aligned} \quad (10.7)$$

Sau đó, thay (10.7) vào (10.1), ta được

$$\begin{aligned} f(X) &= \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \frac{1}{4} (\beta_0 + \beta_1 x_1 + \beta_2 x_2)^2 + \frac{1}{4} (\beta_0 + \beta_1 x_1 - \beta_2 x_2)^2 \\ &= \frac{1}{4} (X_1 + X_2)^2 + \frac{1}{4} (X_1 - X_2)^2 \\ &= X_1 X_2. \end{aligned} \quad (10.8)$$

Vì vậy, tổng của hai phép biến đổi phi tuyến của các hàm tuyến tính có thể tạo ra sự tương tác! Trên thực tế, chúng ta sẽ không sử dụng hàm bậc hai cho $g(z)$, vì chúng ta sẽ luôn nhận được một đa thức bậc hai trong các tọa độ ban đầu $x_1, \dots, x_p$. Các hàm kích hoạt sigmoid hoặc ReLU không có hạn chế như vậy.

Việc lắp đặt mạng nơ-ron yêu cầu ước tính các tham số chưa biết trong (10.1). Đối với phản hồi định lượng, thông thường tổn thất bình phương sai số được sử dụng, do đó các tham số được chọn để giảm thiểu

$$\sum_{i=1}^N (y_i - f(x_i))^2. \quad (10.9)$$

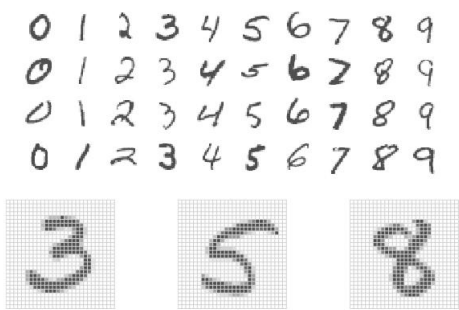
Chi tiết về cách thực hiện phép tối thiểu hóa này được trình bày trong Mục 10.7.

10.2 Mạng nơ-ron đa lớp

Các mạng nơ-ron hiện đại thường có nhiều hơn một lớp ẩn, và thường có nhiều đơn vị trên mỗi lớp. Về lý thuyết, một lớp ẩn duy nhất với số lượng lớn các đơn vị có khả năng xấp xỉ hầu hết các hàm. Tuy nhiên, nhiệm vụ học tập để tìm ra một giải pháp tốt sẽ dễ dàng hơn nhiều với nhiều lớp, mỗi lớp có kích thước vừa phải.

Chúng tôi sẽ minh họa một mạng lưới dày đặc lớn trên tập dữ liệu chữ số viết tay **MNIST** nổi tiếng và được công khai.¹ Hình 10.3 hiển thị các ví dụ về các chữ số này. Ý tưởng là xây dựng một mô hình để phân loại các hình ảnh vào lớp chữ số chính xác từ 0 đến 9. Mỗi hình ảnh có $p = 28 \times 28 = 784$ pixel, mỗi pixel là một giá trị thang độ xám tám bit nằm giữa 0 và 255, đại diện cho các chữ số.

¹Xem LeCun, Cortes và Burges (2010) "Cơ sở dữ liệu MNIST về chữ số viết tay", có sẵn tại <http://yann.lecun.com/exdb/mnist>.



HÌNH 10.3. Ví dụ về các chữ số viết tay từ bộ dữ liệu MNIST . Mỗi hình ảnh thang độ xám có 28 × 28 pixel, mỗi pixel là một số tám bit (0-255) biểu thị độ tối của pixel đó. 3, 5 và 8 pixel đầu tiên được phóng to để hiển thị 784 giá trị pixel riêng lẻ của chúng.

Lượng tương đối của chữ số được viết trong ô vuông nhỏ đó.² Các pixel này được lưu trữ trong vectơ đầu vào X (ví dụ, theo thứ tự cột). Đầu ra là nhãn lớp, được biểu diễn bằng một vectơ $Y = (Y_0, Y_1, \dots, Y_9)$ gồm 10 biến giả, với giá trị 1 ở vị trí tương ứng với nhãn và các giá trị 0 ở những vị trí khác. Trong cộng đồng học máy, điều này được gọi là mã hóa one-hot. Có 60.000 hình ảnh huấn luyện và 10.000 hình ảnh kiểm tra.

mã hóa
one-hot

Xét về mặt lịch sử, các bài toán nhận dạng chữ số chính là chất xúc tác thúc đẩy sự phát triển của công nghệ mạng nơ-ron vào cuối những năm 1980 tại Phòng thí nghiệm AT&T Bell và các nơi khác. Các nhiệm vụ nhận dạng mẫu kiểu này tương đối đơn giản đối với con người. Hệ thống thị giác chiếm một phần lớn não bộ của chúng ta, và khả năng nhận dạng tốt là một động lực tiến hóa để sinh tồn. Những nhiệm vụ này không hề đơn giản đối với máy móc, và phải mất hơn 30 năm để hoàn thiện kiến trúc mạng nơ-ron nhằm đạt được hiệu suất tương đương với con người.

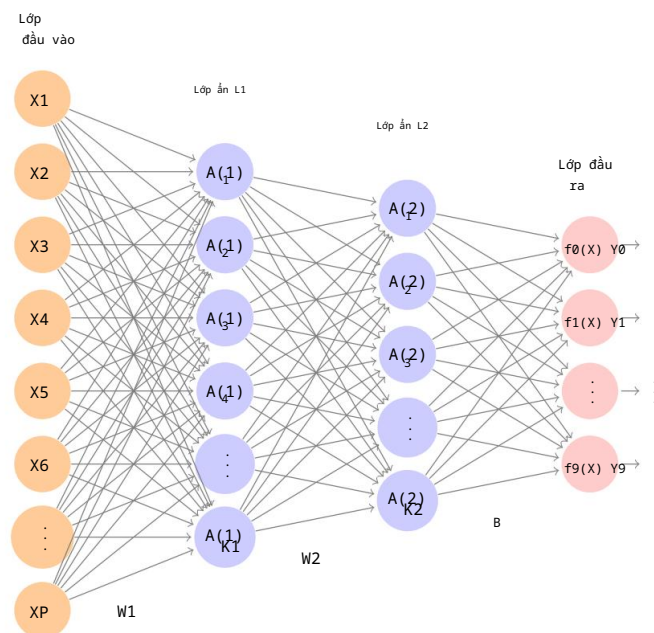
Hình 10.4 thể hiện kiến trúc mạng đa lớp hoạt động hiệu quả trong việc giải quyết bài toán phân loại chữ số. Nó khác với Hình 10.1 ở một số điểm.

cách thức:

- Nó có hai lớp ẩn L1 (256 đơn vị) và L2 (128 đơn vị) thay vì một. Sau này chúng ta sẽ xem xét một mạng có bảy lớp ẩn.
- Nó có mười biến đầu ra, thay vì một. Trong trường hợp này, mười biến này thực sự đại diện cho một biến định tính duy nhất và do đó khá phụ thuộc lẫn nhau. (Chúng tôi đã đánh số chúng theo lớp chữ số từ 0-9 thay vì 1-10 để dễ hiểu hơn.) Nói chung hơn, trong học đa nhiệm, người ta có thể dự đoán các phản hồi khác nhau đồng thời với một mạng duy nhất; tất cả chúng đều có tiếng nói trong việc hình thành các lớp ẩn.
- Hàm mất mát được sử dụng để huấn luyện mạng được thiết kế riêng cho nhiệm vụ phân loại đa lớp.

đa nhiệm
học hồi

²Trong quá trình chuyển đổi từ tín hiệu tương tự sang tín hiệu số, chỉ một phần của chữ số được viết có thể nằm trong ô vuông đại diện cho một điểm ảnh cụ thể.



HÌNH 10.4. Sơ đồ mạng nơ-ron với hai lớp ẩn và nhiều đầu ra, phù hợp với bài toán chữ số viết tay **MNIST**. Lớp đầu vào có $p = 784$ đơn vị, hai lớp ẩn $K1 = 256$ và $K2 = 128$ đơn vị tương ứng, và lớp đầu ra có 10 đơn vị. Cùng với các hệ số chặn (được gọi là độ lệch trong cộng đồng học sâu), mạng này có 235.146 tham số (được gọi là trọng số).

lớp ẩn đầu tiên giống như trong (10.2), với

$$\begin{aligned} A_k^{(1)} &= h_k^{(1)}(X) \\ &= g(w_{k0}^{(1)} + \sum_{j=1}^p w_{kj}^{(1)} X_j) \end{aligned} \quad (10.10)$$

với $k = 1, \dots, K1$. Lớp ẩn thứ hai xử lý các kích hoạt $A^{(1)}$ của lớp ẩn đầu tiên như đầu vào và tính toán các kích hoạt mới

$$A_k^{(2)} = h_k^{(2)}(X) = g(w_{k0}^{(2)} + \sum_{k=1}^{K1} w_{kk}^{(2)} A_k^{(1)}) \quad (10.11)$$

với $k = 1, \dots, K2$. Lưu ý rằng mỗi kích hoạt trong lớp thứ hai $A^{(2)} = h^{(2)}(X)$ là một hàm của vectơ đầu vào X . Điều này là do trong khi chúng rõ ràng là một hàm của các kích hoạt $A^{(1)}$ từ lớp L1, các lớp này lần lượt là các hàm của X . Điều này cũng đúng với nhiều lớp ẩn hơn. Do đó, thông qua một chuỗi các phép biến đổi, mạng có thể xây dựng các phép biến đổi khá phức tạp của X , cuối cùng được đưa vào lớp đầu ra dưới dạng các đặc trưng.

Chúng tôi đã giới thiệu thêm ký hiệu chỉ số trên như $h^{(2)}(X)$ và $w^{(2)}$ trong (10.10) và (10.11) để chỉ ra các kích hoạt và trọng số (hệ số) thuộc về lớp nào, trong trường hợp này là lớp 2. Ký hiệu $W1$ trong Hình-
trọng lượng

Hình 10.4 biểu thị toàn bộ ma trận trọng số được lấy từ đầu vào.
lớp tới lớp ẩn đầu tiên L1. Ma trận này sẽ có kích thước $785 \times 256 = 200.960$.
các phần tử; có 785 chữ không phải 784 vì chúng ta phải tính đến
Hệ số chặn hoặc hệ số sai lệch.³

thiên vị

Mỗi phần tử $A(1)_k$ các đầu vào được đưa đến lớp ẩn thứ hai L2 thông qua ma trận của
Trọng lượng W_2 có kích thước $257 \times 128 = 32.896$.

Giờ chúng ta đến lớp đầu ra, nơi chúng ta có mười phản hồi.
hơn một. Bước đầu tiên là tính toán mười mô hình tuyến tính khác nhau tương tự.
đến mô hình đơn lẻ của chúng tôi (10.1),

$$\begin{aligned} Z_m &= \beta m_0 + \sum_{h=1}^{K_2} \beta m_h(2) (X) \\ &= \beta m_0 + \sum_{h=1}^{K_2} \beta m_h(2) , \end{aligned} \tag{10.12}$$

với $m = 0, 1, \dots, 9$. Ma trận B lưu trữ tất cả $129 \times 10 = 1.290$ giá trị này.
trọng lượng.

Nếu đây đều là các phản hồi định lượng riêng biệt, chúng ta chỉ cần thiết lập
mỗi $f_m(X) = Z_m$ và được thực hiện. Tuy nhiên, chúng tôi muốn các ước tính của mình
biểu diễn xác suất lớp $f_m(X) = \Pr(Y = m|X)$, giống như trong hồi quy logistic đa thức ở
Mục 4.3.5. Vì vậy, chúng tôi sử dụng hàm kích hoạt softmax đặc biệt (xem (4.13) trên trang
145),

softmax

$$f_m(X) = \Pr(Y = m|X) = \frac{e^{Z_m}}{\sum_{n=0}^9 e^{Z_n}} , \tag{10.13}$$

với $m = 0, 1, \dots, 9$. Điều này đảm bảo rằng 10 số này hoạt động như các xác suất (không
âm và tổng bằng một). Mặc dù mục tiêu là xây dựng
là một bộ phân loại, mô hình của chúng tôi thực sự ước tính xác suất cho mỗi trong số 10.
các lớp. Sau đó, bộ phân loại sẽ gán hình ảnh vào lớp có độ chính xác cao nhất.
xác suất.

Để huấn luyện mạng này, vì phản hồi mang tính định tính, chúng ta tìm kiếm các hệ số...
ước tính hiệu quả nhằm giảm thiểu log-likelihood đa thức âm.

$$-\sum_{i=1}^n \sum_{m=0}^9 y_{im} \log(f_m(x_i)) , \tag{10.14}$$

còn được gọi là entropy chéo. Đây là sự tổng quát hóa của tiêu chí (4.5) cho hồi quy
logistic hai lớp. Chi tiết về cách tối thiểu hóa mục tiêu này được trình bày trong Phần
10.7. Nếu phản hồi là định lượng, chúng ta
thay vào đó sẽ giảm thiểu tổn thất bình phương sai số như trong (10.9).

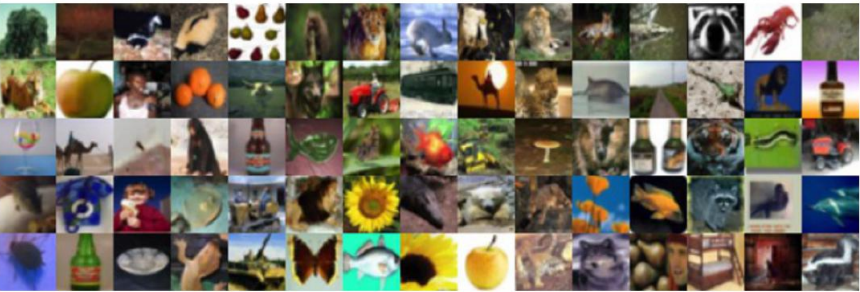
đi qua-
entropy

Bảng 10.1 so sánh hiệu suất kiểm thử của mạng nơ-ron với
hai mô hình đơn giản được trình bày trong Chương 4 sử dụng quyết định tuyến tính
Các phương pháp được sử dụng: hồi quy logistic đa biến và phân tích phân biệt tuyến tính.
Ưu điểm của mạng nơ-ron so với cả hai phương pháp tuyến tính này là:
Ảnh hưởng: mạng với phương pháp điều chỉnh dropout đạt được tỷ lệ lỗi kiểm thử cao.
dưới 2% trên 10.000 hình ảnh thử nghiệm. (Chúng tôi mô tả phương pháp điều chỉnh dropout trong...)
(Mục 10.7.3.) Trong Mục 10.9.2 của bài thực hành, chúng tôi trình bày mã để hiệu chỉnh.
Mô hình này chạy chỉ trong hơn hai phút trên máy tính xách tay.

³Việc sử dụng “trọng số” cho các hệ số và “độ lệch” cho các hệ số chặn w_0 trong (10.2) là
phổ biến trong cộng đồng học máy; cách sử dụng thiên kiến này không nên bị nhầm lẫn với
Việc sử dụng thuật ngữ “sai lệch-phương sai” ở những phần khác trong cuốn sách này.

Phương pháp	Lỗi kiểm thử
Mạng nơ-ron + Điều chỉnh Ridge	2,3%
Mạng nơ-ron + Điều chỉnh Dropout	1,8%
Hồi quy Logistic Đa thức	7,2%
Phân tích phân biệt tuyến tính	12,7%

BẢNG 10.1. Tỷ lệ lỗi kiểm thử trên tập dữ liệu MNIST, đối với mạng nơ-ron có hai các hình thức điều chỉnh, cũng như hồi quy logistic đa thức và phân tích phân biệt tuyến tính. Trong ví dụ này, sự phức tạp bổ sung của mạng nơ-ron. Điều này dẫn đến sự cải thiện đáng kể về tỷ lệ lỗi trong quá trình kiểm thử.



HÌNH 10.5. Một số hình ảnh mẫu từ cơ sở dữ liệu CIFAR100: một bộ sưu tập các hình ảnh. Những hình ảnh tự nhiên từ cuộc sống thường nhật, với 100 loại hình khác nhau được thể hiện.

Cộng số lượng hệ số trong W1, W2 và B, ta được 235.146 trong tất cả, nhiều hơn 33 lần con số $785 \times 9 = 7,065$ cần thiết cho đa thức. Hồi quy logistic. Hãy nhớ rằng có 60.000 hình ảnh trong tập dữ liệu huấn luyện. Mặc dù tập dữ liệu huấn luyện này có vẻ lớn, nhưng thực tế có gần gấp bốn lần. Số lượng hệ số trong mô hình mạng nơ-ron bằng với số lượng quan sát. Tập dữ liệu huấn luyện! Để tránh hiện tượng quá khớp, cần có một số phương pháp điều chỉnh (regularization). Trong trường hợp này Ví dụ, chúng tôi đã sử dụng hai hình thức điều chỉnh: điều chỉnh theo đường gờ, mà Tương tự như hồi quy Ridge từ Chương 6 và chuẩn hóa dropout. Chúng tôi sẽ thảo luận cả hai hình thức điều chỉnh trong Phần 10.7.

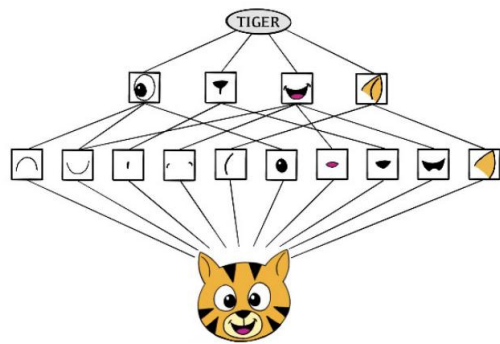
bỏ học

10.3 Mạng nơ-ron tích chập

Mạng nơ-ron đã hồi sinh mạnh mẽ vào khoảng năm 2010 với những thành công lớn trong phân loại hình ảnh. Vào khoảng thời gian đó, các cơ sở dữ liệu khổng lồ về hình ảnh được gắn nhãn đang được xây dựng. tích lũy dần, với số lượng lớp học ngày càng tăng. Hình 10.5 cho thấy 75 hình ảnh được lấy từ cơ sở dữ liệu CIFAR100.4 Cơ sở dữ liệu này bao gồm 60.000 hình ảnh được dán nhãn theo 20 siêu lớp (ví dụ: động vật có vú sống dưới nước), với năm lớp con trong mỗi siêu lớp (hải ly, cá heo, rái cá, hải cẩu, cá voi). Mỗi Hình ảnh có độ phân giải 32×32 pixel, với ba số tám bit trên mỗi pixel. Điểm ảnh đại diện cho màu đỏ, xanh lá cây và xanh dương. Các con số cho mỗi hình ảnh là được sắp xếp thành một mảng ba chiều gọi là bản đồ đặc trưng. Hai phần đầu tiên

bản đồ đặc trưng

4Xem Chương 3 của Krizhevsky (2009) "Học nhiều lớp đặc trưng từ hình ảnh nhỏ", có sẵn tại <https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf>.



HÌNH 10.6. Sơ đồ minh họa cách mạng nơ-ron tích chập phân loại hình ảnh một con hổ. Mạng này nhận đầu vào là hình ảnh và xác định các đặc điểm cục bộ. Sau đó, nó kết hợp các đặc điểm cục bộ để tạo ra các đặc điểm phức hợp, trong ví dụ này bao gồm mắt và tai. Các đặc điểm phức hợp này được sử dụng để xuất ra nhãn “hổ”.

Các trục là trục không gian (cả hai đều có 32 chiều), và trục thứ ba là trục kênh, 5
biểu thị ba màu. Có một tập huấn luyện gồm 50.000 hình ảnh và một tập kiểm tra gồm 10.000 hình ảnh.

Một họ mạng nơ-ron tích chập (CNN) đặc biệt đã được phát triển để phân loại các hình ảnh như thể này, và đã cho thấy sự thành công ngoạn mục trên nhiều bài toán khác nhau. CNN mô phỏng ở một mức độ nào đó cách con người phân loại hình ảnh, bằng cách nhận diện các đặc điểm hoặc mẫu cụ thể ở bất kỳ đâu trong hình ảnh để phân biệt từng lớp đối tượng cụ thể. Trong phần này, chúng tôi sẽ trình bày tổng quan ngắn gọn về cách chúng hoạt động.

Hình 10.6 minh họa ý tưởng đằng sau mạng nơ-ron tích chập trên hình ảnh hoạt hình của một con hổ.6 Mạng này thực

tiên xác định các đặc điểm cấp thấp trong hình ảnh đầu vào, chẳng hạn như các cạnh nhỏ, các mảng màu, v.v. Sau đó, các đặc điểm cấp thấp này được kết hợp để tạo thành các đặc điểm cấp cao hơn, chẳng hạn như các bộ phận của tai, mắt, v.v. Cuối cùng, sự hiện diện hoặc vắng mặt của các đặc điểm cấp cao hơn này góp phần vào xác suất của bất kỳ lớp đầu ra nào.

Mạng nơ-ron tích chập xây dựng cấu trúc phân cấp này như thế nào? Nó kết hợp hai loại lớp ẩn chuyên biệt, được gọi là lớp tích chập và lớp gộp . Lớp tích chập tìm kiếm các mẫu nhỏ trong ảnh, trong khi lớp gộp giảm kích thước các mẫu này để chọn ra một tập con nổi bật. Để đạt được kết quả tiên tiến nhất, các kiến trúc mạng nơ-ron hiện đại sử dụng rất nhiều lớp tích chập và lớp gộp.

Tiếp theo, chúng ta sẽ mô tả các lớp tích chập và lớp gộp.

10.3.1 Lớp tích chập

Lớp tích chập được tạo thành từ một số lượng lớn các bộ lọc tích chập, mỗi bộ lọc là một phần tử con. lớp tích chập

5. Thuật ngữ "kênh" được lấy từ tài liệu về xử lý tín hiệu. Mỗi kênh là một nguồn thông tin riêng biệt.

6. Cảm ơn Elena Tuzhilina đã tạo ra sơ đồ và <https://www.cartooning4kids.com/> Xin phép sử dụng hình ảnh chú hổ hoạt hình.

bộ lọc tích chập

Trong đó, có một khuôn mẫu xác định xem một đặc điểm cục bộ cụ thể có xuất hiện trong ảnh hay không. Bộ lọc tích chập dựa trên một phép toán rất đơn giản, được gọi là tích chập, về cơ bản là nhân các phần tử ma trận lặp đi lặp lại rồi cộng các kết quả lại với nhau.

Để hiểu cách hoạt động của bộ lọc tích chập, hãy xem xét một ví dụ rất đơn giản về ảnh 4×3 :

Ảnh gốc =

abc
định nghĩa
ghi
jkl

.

Bây giờ hãy xem xét một bộ lọc 2×2 có dạng

Bộ lọc tích chập =

aβ
γδ

.

Khi ta tích chập ảnh với bộ lọc, ta sẽ nhận được kết quả.

Ảnh tích chập =

aa + bβ + dγ + eδ ba + cβ + eγ + fδ
da + eβ + gγ + hδ ea + fβ + hγ + iδ
ga + hβ + jγ + kδ ha + iβ + kγ + lδ

.

Ví dụ, phần tử ở góc trên bên trái được tạo ra bằng cách nhân mỗi phần tử trong bộ lọc 2×2 với phần tử tương ứng trong phần 2×2 ở góc trên bên trái của ảnh, rồi cộng các kết quả lại. Các phần tử khác được tạo ra theo cách tương tự: bộ lọc tích chập được áp dụng cho mọi ma trận con 2×2 của ảnh gốc để thu được ảnh tích chập. Nếu một ma trận con 2×2 của ảnh gốc giống với bộ lọc tích chập, thì nó sẽ có giá trị lớn trong ảnh tích chập; ngược lại, nó sẽ có giá trị nhỏ.

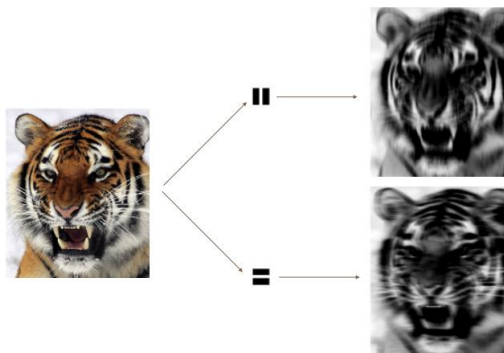
Do đó, ảnh tích chập làm nổi bật các vùng của ảnh gốc giống với bộ lọc tích chập. Chúng ta đã sử dụng 2×2 làm ví dụ; nói chung, các bộ lọc tích chập là các mảng nhỏ 1×2 , với các số nguyên dương nhỏ không nhất thiết phải bằng nhau.

Và
1 2

Hình 10.7 minh họa việc áp dụng hai bộ lọc tích chập vào ảnh con hổ có kích thước 192×179 , được hiển thị ở phía bên trái.8 Mỗi bộ lọc tích chập là một ảnh 15×15 chứa chủ yếu là số 0 (màu đen), với một dải hẹp gồm các số 1 (màu trắng) được định hướng theo chiều dọc hoặc chiều ngang trong ảnh. Khi mỗi bộ lọc được tích chập với ảnh con hổ, các vùng của con hổ giống với bộ lọc (tức là có sọc ngang hoặc dọc hoặc các cạnh) sẽ được gán giá trị lớn, và các vùng của con hổ không giống với đặc điểm đó sẽ được gán giá trị nhỏ. Ảnh đã được tích chập được hiển thị ở phía bên phải. Chúng ta thấy rằng bộ lọc sọc ngang chọn ra các sọc ngang và các cạnh trong ảnh gốc, trong khi bộ lọc sọc dọc chọn ra các sọc dọc và các cạnh trong ảnh gốc.

7. Ảnh đã được tích chập nhỏ hơn ảnh gốc vì kích thước của nó được xác định bởi số lượng các ma trận con 2×2 trong ảnh gốc. Lưu ý rằng 2×2 là kích thước của bộ lọc tích chập. Nếu chúng ta muốn ảnh đã được tích chập có cùng kích thước với ảnh gốc, thì có thể áp dụng kỹ thuật đệm (padding).

8 Hình ảnh con hổ được sử dụng trong Hình 10.7-10.9 được lấy từ nguồn công cộng.
Nguồn ảnh: <https://www.needpix.com/>.



HÌNH 10.7. Các bộ lọc tích chập tìm các đặc điểm cục bộ trong ảnh, chẳng hạn như các cạnh và hình dạng nhỏ. Chúng ta bắt đầu với hình ảnh con hổ được hiển thị ở bên trái và áp dụng hai bộ lọc tích chập nhỏ ở giữa. Các hình ảnh được tích chập làm nổi bật các khu vực trong hình ảnh gốc nơi tìm thấy các chi tiết tương tự như các bộ lọc. Cụ thể, ảnh tích chập phía trên làm nổi bật các sọc dọc của hổ, trong khi ảnh tích chập phía dưới làm nổi bật các sọc ngang của hổ. Ta có thể coi ảnh gốc như lớp đầu vào trong mạng nơ-ron tích chập, và các ảnh tích chập như các đơn vị trong lớp ẩn đầu tiên.

Chúng tôi đã sử dụng một ảnh lớn và hai bộ lọc lớn trong Hình 10.7 để minh họa. Đối với cơ sở dữ liệu CIFAR100, mỗi ảnh có 32×32 pixel màu, và chúng tôi sử dụng bộ lọc tích chập 3×3 .

Trong lớp tích chập, chúng ta sử dụng một tập hợp các bộ lọc để chọn ra nhiều cạnh và hình dạng khác nhau trong ảnh. Việc sử dụng các bộ lọc được định sẵn theo cách này là thông lệ tiêu chuẩn trong xử lý ảnh. Ngược lại, với mạng CNN, các bộ lọc được học cho nhiệm vụ phân loại cụ thể. Chúng ta có thể coi trọng số của bộ lọc như các tham số đi từ lớp đầu vào đến lớp ẩn, với một đơn vị ẩn cho mỗi pixel trong ảnh đã được tích chập.

Thực tế đúng là như vậy, mặc dù các tham số được cấu trúc và ràng buộc rất chặt chẽ (xem Bài tập 4 để biết thêm chi tiết). Chúng hoạt động trên các vùng cục bộ trong ảnh đầu vào (do đó có nhiều giá trị bằng không về mặt cấu trúc), và cùng một trọng số trong một bộ lọc nhất định được sử dụng lại cho tất cả các vùng có thể có trong ảnh (do đó các trọng số bị ràng buộc).⁹

Chúng tôi sẽ cung cấp thêm một số chi tiết.

- Vì ảnh đầu vào là ảnh màu, nên nó có ba kênh được biểu diễn bằng bản đồ đặc trưng ba chiều (mảng). Mỗi kênh là một bản đồ đặc trưng hai chiều (32×32) – một cho màu đỏ, một cho màu xanh lá cây và một cho màu xanh lam. Một bộ lọc tích chập đơn cũng sẽ có ba kênh, một kênh cho mỗi màu, mỗi kênh có kích thước 3×3 , với trọng số bộ lọc có thể khác nhau. Kết quả của ba phép tích chập được cộng lại để tạo thành một bản đồ đặc trưng đầu ra hai chiều. Lưu ý rằng tại thời điểm này, thông tin màu sắc đã được sử dụng và không được truyền đến các lớp tiếp theo ngoại trừ thông qua vai trò của nó trong phép tích chập.

9. Trong những năm đầu của mạng nơ-ron, điều này từng được gọi là chia sẻ trọng lượng.